

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
School of Computer Science and Engineering



**DEVELOPMENT OF AN INTELLIGENT WEB-BASED
FRAMEWORK FOR EARLY DETECTION OF
BURNOUT IN THE INDUSTRIAL WORKFORCE**

*A Microservice Approach Using the Copenhagen Burnout Inventory
and Random Forest Regression*

By

Nguyen Phuc Dat

Student ID: ITITIU22028

Instructor

Dang Tam Nhan

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science

Ho Chi Minh City, Vietnam

2026

**DEVELOPMENT OF AN INTELLIGENT WEB-BASED FRAMEWORK FOR
EARLY DETECTION OF BURNOUT IN THE INDUSTRIAL WORKFORCE**

APPROVED BY:

Dang Tam Nhan, Supervisor

Committee Member

Committee Member

THESIS COMMITTEE

ACKNOWLEDGMENTS

I am grateful to my supervisor, **Dang Tam Nhan**, for his guidance, feedback, and encouragement throughout this thesis. His expertise in AI and software engineering helped shape the direction of the work, and his patience during every revision cycle was invaluable.

I would also like to thank the faculty of the **School of Computer Science and Engineering, International University – Vietnam National University Ho Chi Minh City**, for providing an academic environment that demanded independent research and critical thinking. The foundations laid during my undergraduate studies made this project possible.

Thanks are due to my classmates and friends for discussions and support throughout the process. In particular, **Tran Luu Hong Phuong** and **Luong Quang Huy** provided early feedback that surfaced usability problems and architectural issues I might otherwise have missed.

I am deeply thankful to my family for their support and encouragement during my studies. Their belief in me sustained the effort required to see this project through.

Finally, I acknowledge the open-source community and the developers behind the frameworks used in this thesis, including **React**, **Next.js**, **Node.js**, **Express**, **FastAPI**, and **scikit-learn**, without which a system of this scope would not have been practical.

Nguyen Phuc Dat

Ho Chi Minh City, May 7, 2026

Contents

1	Introduction	1
1.1	Background	1
1.2	Motivating Scenario and Use Cases	2
1.3	Problem Statement	3
1.4	Scope and Objectives	4
1.5	Research Questions	5
1.6	Assumptions and Proposed Solution	5
1.7	Thesis Structure	6
2	Literature Review	9
2.1	Occupational Burnout	9
2.1.1	Definition and Impact on the Industrial Workforce	9
2.1.2	Burnout Measurement Instruments	10
2.1.3	The Copenhagen Burnout Inventory in This Thesis	10
2.2	Machine Learning for Health Prediction	11
2.2.1	Supervised Regression	11
2.2.2	Linear Regression	11
2.2.3	Random Forest Regression	12
2.2.4	Model Evaluation Metrics	13
2.3	Web Application Architectures	14
2.3.1	Microservices vs. Monolithic Architecture	14
2.3.2	RESTful API Design	14
2.4	Related Technologies	15
2.4.1	React and Next.js for Frontend Development	15

2.4.2	Node.js and Express for Backend Services	15
2.4.3	FastAPI for Machine Learning Serving	16
2.5	Related Work	16
3	Methodology and System Design	18
3.1	System Architecture Overview	18
3.2	Database Design	19
3.2.1	MySQL Schema	19
3.2.2	MongoDB Collections	20
3.3	Backend Design	21
3.3.1	Layered Architecture	21
3.3.2	Authentication and Authorisation	23
3.3.3	Quiz Scoring Pipeline	23
3.3.4	Alert Rules	24
3.4	AI Microservice Design	25
3.4.1	Dataset Description	25
3.4.2	Feature Selection Justification	26
3.4.3	Data Preprocessing	26
3.4.4	Model Selection	27
3.4.5	FastAPI Service Architecture	27
3.5	Frontend Design	28
3.5.1	Technology Stack and Design Decisions	28
3.5.2	Route Structure and Navigation	29
3.5.3	Design System	30
3.5.4	Data Flow Pattern	31

4	Implementation and Results	33
4.1	Development Environment	33
4.2	Backend Implementation	33
4.2.1	Project Structure and Configuration	33
4.2.2	Authentication Flow Implementation	34
4.2.3	Quiz Scoring Implementation	35
4.2.4	Dashboard Aggregation	35
4.2.5	Alert System Implementation	36
4.2.6	API Endpoint Summary	36
4.3	AI Service Implementation	37
4.3.1	Model Training Results	37
4.3.2	FastAPI Prediction Endpoint	40
4.3.3	Integration with Express Backend	41
4.4	Frontend Implementation	41
4.4.1	Authentication Pages	41
4.4.2	Employee Portal	42
4.4.3	Admin Portal	46
4.4.4	Reusable UI Components	51
4.4.5	State Management with TanStack Query	51
5	Discussion and Evaluation	53
5.1	Model Performance Analysis	53
5.2	Functional Testing	54
5.3	System Feature Evaluation	56
5.4	Answering the Research Questions	57

5.5	Comparison with Related Work	58
6	Conclusion and Future Work	59
6.1	Conclusion	59
6.2	Contributions	60
6.3	Achievement of Objectives	60
6.4	Limitations	61
6.5	Future Work	62
	List of References	64
	Appendices	68
	Appendix A: Complete API Reference	68
	Appendix B: Database Schema	71
	Appendix C: CBI Questionnaire	77

List of Tables

1	Comparison with related work.	17
2	Dataset columns used for model training.	25
3	Primary design tokens.	31
4	Development tools and versions.	33
5	Summary of backend API endpoints.	37
6	Model performance comparison.	38
7	Functional test cases and results.	54
8	Evaluation of system objectives.	56
9	Complete API endpoint reference.	68

List of Figures

1	Structure of the thesis.	8
2	High-level system architecture.	18
3	Entity-relationship diagram of the MySQL database.	19
4	Layered architecture of the Express backend.	22
5	Submitting Quiz results pipeline.	24
6	Architecture of the FastAPI(Python) AI microservice.	28
7	Application route structure and navigation flow.	30
8	Random Forest feature importance scores.	38
9	Distribution of Burn Rate values in the training dataset.	39
10	Pearson correlation heatmap of features and Burn Rate.	39
11	Actual vs. predicted burn rate for both models.	40
12	Residual analysis for the Random Forest model.	40
13	Login page.	42
14	Registration page.	42
15	Employee dashboard with burn rate gauge and trend chart.	43
16	CBI assessment page - Step 1: Personal Burnout.	44
17	Assessment results page with score cards and support resources.	45
18	Quiz submission history with expandable response detail.	46
19	Admin command centre dashboard.	47
20	Department detail - Employees tab with filters and sorting.	48
21	Department detail - Analytics tab with burnout and KPI visualisations.	48
22	Admin employee detail with burnout and KPI analysis.	49
23	Admin alerts inbox with filtering.	50

24	Help resource management with CRUD operations.	50
----	--	----

ABSTRACT

Occupational burnout imposes measurable costs on industrial workplaces through absenteeism, reduced output, and elevated accident rates. Conventional detection methods rely on periodic surveys or supervisor observation, both of which are inherently reactive: by the time burnout becomes visible, its organisational impact has already accumulated.

This thesis presents the design and implementation of an intelligent web-based framework for early burnout detection in industrial settings. The system integrates a validated psychometric instrument, the **Copenhagen Burnout Inventory (CBI)**, with a **Random Forest** regression model that predicts a continuous burnout score from three features: employee designation, resource allocation, and a mental fatigue score derived from CBI responses.

The framework follows a microservice architecture comprising three components. A **React/Next.js** frontend provides role-differentiated portals for employees (self-assessment, trend tracking, support resources) and administrators (department heatmaps, risk distribution analytics, KPI correlation charts, automated alert inbox). A **Node.js/Express** backend manages authentication, quiz scoring, risk classification, alert generation, and dashboard aggregation, backed by **MySQL** for relational data and **MongoDB** for the quiz definition and raw submissions. A **Python/FastAPI** microservice loads the serialised model at startup and exposes a prediction endpoint, maintaining a clean separation between the machine learning component and the application backend.

The Random Forest model was trained on approximately 22,000 records from a public Kaggle dataset. On the held-out test set, it achieves an R^2 of 0.90, an MAE of 0.048, and an RMSE of 0.061, outperforming the Linear Regression baseline across

all three metrics. Feature importance analysis indicates that the mental fatigue score accounts for approximately 81% of predictive weight, consistent with its role as the primary psychometric input.

Several limitations apply. HR profile data for designation and resource allocation are synthetically generated rather than sourced from a real factory system. The model relies on three features and is evaluated against a single public dataset, which constrains generalisability across industries and organisational contexts. No formal usability study was conducted. Within these constraints, the system demonstrates a practical architecture for combining psychometric assessment, machine learning prediction, and organisational analytics into a unified platform for proactive burnout management.

Keywords: framework, employee's mental health, employee burnout detection, Copenhagen Burnout Inventory, Random Forest regression, Linear regression, microservice architecture, industrial workforce, web-based health monitoring.

CHAPTER I

INTRODUCTION

1.1 Background

Industrial workers operate under conditions that make occupational burnout particularly prevalent. Factory environments routinely combine physically demanding tasks, rigid shift schedules, production quotas, and limited autonomy. The World Health Organisation recognised burnout as an occupational phenomenon in the eleventh revision of the International Classification of Diseases (ICD-11), defining it as a syndrome resulting from chronic workplace stress that has not been successfully managed [1]. Its three characteristic dimensions, emotional exhaustion, depersonalisation, and reduced personal accomplishment, manifest in measurable organisational outcomes: declining productivity, increased absenteeism, elevated turnover, and higher healthcare costs [2].

Stress-related conditions, including burnout, are estimated to cost organisations hundreds of billions of dollars annually in lost productivity and medical expenses [6]. In manufacturing and industrial sectors the consequences extend beyond individual welfare: fatigued workers are disproportionately associated with safety incidents, quality defects, and equipment damage. Despite these risks, most organisations still rely on ad-hoc supervisor observation and infrequent surveys to assess employee mental health, an approach that is inherently reactive rather than preventive.

Advances in machine learning and web development have created practical opportunities to shift burnout management toward proactive, continuous monitoring. By pairing validated psychometric instruments with predictive models and real-time

dashboards, it is feasible to build systems that estimate burnout risk at both the individual and organisational level, alerting management before problems escalate while also equipping employees with self-monitoring tools.

1.2 Motivating Scenario and Use Cases

Consider a mid-sized manufacturing plant with 200 workers across six departments operating rotating day and night shifts. With three HR staff managing attendance, grievances, and benefits concurrently, proactive mental health monitoring is not feasible under current practice. Burnout is only surfaced when a supervisor notices persistent signs such as chronic fatigue or declining output and escalates to HR. At that point, weeks or months of deteriorating mental health have already passed and the department has likely absorbed measurable productivity losses.

The system proposed in this thesis replaces this reactive workflow with a continuous, data-driven one. Two use cases drive the design:

1. **Employee self-assessment and trend tracking.** Each worker periodically completes a 13-question burnout assessment through a web portal. The system immediately scores the quiz, invokes the ML prediction endpoint, and displays the predicted burn rate as a gauge visualization. Historical results are plotted as trend charts, allowing the worker to monitor changes over time. When risk is classified as high or critical, curated mental health resources are surfaced on the results page.
2. **HR department-level monitoring and alerts.** The admin portal aggregates assessment data across the workforce. A department heatmap visualises average burn rates by organisational unit, enabling HR to identify high-stress areas. Automated alerts are generated when an individual's burn rate exceeds a

critical threshold or increases sharply between consecutive assessments. Drill-down views present anonymised employee-level detail, balancing privacy with the need for targeted intervention.

1.3 Problem Statement

Despite the well-documented impact of burnout on industrial productivity and worker welfare, several gaps remain in current organisational practice:

1. **Detection is reactive.** Most organisations identify burnout only after its symptoms, including absenteeism, errors, and resignations, have already materialised. No systematic early-warning mechanism is in place.
2. **Psychometric data is underutilised.** Validated instruments such as the Copenhagen Burnout Inventory are freely available, yet they are rarely embedded in continuous monitoring workflows. When surveys are administered, results are typically analysed manually and in isolation rather than fed into an integrated system.
3. **No predictive capability.** Traditional approaches report current symptom levels but do not estimate future risk. Machine learning can address this gap by learning patterns from historical data and projecting burn rates from a small set of measurable inputs.
4. **No organisational-level visibility.** Individual survey results remain isolated. HR managers lack aggregated analytical tools, such as department heatmaps, risk distribution charts, and trend analyses, that would support data-driven workforce management decisions.

This thesis addresses these gaps through an integrated web-based framework

that combines psychometric assessment, machine learning prediction, and multi-level analytics on a single platform.

1.4 Scope and Objectives

The scope of this thesis covers the design, implementation, and evaluation of a desktop web application for early burnout detection in industrial workplaces. The system uses simulated HR data (since real factory records are private and inaccessible), the Copenhagen Burnout Inventory as the psychometric tool, and a Random Forest regression model trained on a public dataset as the predictive engine.

The specific objectives are:

1. Implement a web-based Copenhagen Burnout Inventory (CBI) assessment that turns employee responses into a mental fatigue score ready for machine learning input.
2. Train and evaluate a Random Forest regression model to predict employee burn rate from three features: designation, resource allocation, and mental fatigue score.
3. Build an employee self-monitoring portal with a burn rate gauge, historical trend charts, section score breakdowns, and contextual access to mental health resources.
4. Build an administrative command center with department heatmaps, risk distribution analytics, KPI correlation views, and a drill-down capability for individual employee analysis.
5. Implement an automated alert system that notifies administrators when employees cross critical risk thresholds or show sharp burn rate increases between

assessments.

6. Demonstrate a microservice architecture that cleanly separates the machine learning inference part (Python/FastAPI) from the application backend (Node.js/Express) and the user interface (React/Next.js).

1.5 Research Questions

This thesis investigates the following research questions:

- RQ1.** How effectively can a Random Forest regression model predict employee burn rate using only three input features (designation, resource allocation, and mental fatigue score)?
- RQ2.** Can a web-based system provide actionable burnout insights to both employees (through self-monitoring tools) and HR administrators (through department-level analytics and automated alerts)?
- RQ3.** What are the practical benefits and trade-offs of using a microservice architecture to separate machine learning inference from the application backend in a health monitoring system?

1.6 Assumptions and Proposed Solution

The following assumptions underlie the design of the system:

- **Simulated HR data.** Real factory HR databases are private and inaccessible for academic research. The backend generates synthetic HR profiles (designation, resource allocation, department, shift type) at registration time, drawing values from the same ranges as the training dataset to ensure the ML model receives in-distribution inputs.

- **CBI as a valid burnout measure.** The Copenhagen Burnout Inventory is an established, freely available instrument with demonstrated reliability and validity in occupational health research. Its 13-question excerpt (personal and work-related burnout sections) is used as the sole actively collected data point.
- **Desktop-only usage.** The system targets HR office terminals and factory kiosk stations. No mobile-responsive layout is implemented.
- **Single-language interface.** The user interface is presented in English. Multi-language support is deferred to future work.

The proposed solution is a three-tier web app:

- A **React/Next.js frontend** with two role-based portals (employee and admin), connected to the backend through RESTful API calls, with TanStack Query handling server state.
- A **Node.js/Express backend** that manages authentication (JWT), quiz scoring (CBI with reverse scoring), risk classification, alert generation, dashboard aggregation, and resource management, using MySQL and MongoDB as databases.
- A **Python/FastAPI AI microservice** that loads a pre-trained Random Forest model at startup and exposes a single prediction endpoint. The backend can switch between the real AI service and an in-process mock formula through an environment variable, so each component can be developed and deployed independently.

1.7 Thesis Structure

This thesis is organised into six chapters, as shown in Figure 1.

Chapter I: Introduction covers the background, motivating scenario, problem statement, objectives, research questions, and assumptions.

Chapter II: Literature Review examines the academic foundations of occupational burnout, the Copenhagen Burnout Inventory, machine learning methods for burnout prediction, web application architectures, and related work.

Chapter III: Methodology and System Design describes the overall system architecture, database design, backend design, AI microservice methodology, and frontend design decisions.

Chapter IV: Implementation and Results details the implementation of each component, presents the machine learning training results, and documents the frontend with annotated screenshots.

Chapter V: Discussion and Evaluation analyses model performance, reports functional testing outcomes, evaluates system features against the stated objectives, answers the research questions, and compares the system with related work.

Chapter VI: Conclusion and Future Work summarises the contributions, assesses the achievement of objectives, discusses limitations, and proposes directions for future development.

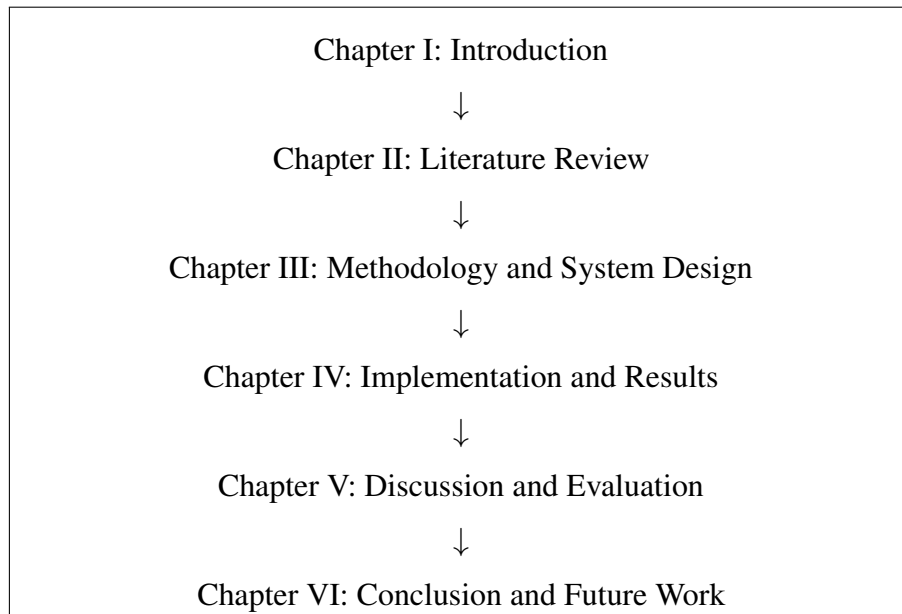


Figure 1: Structure of the thesis.

CHAPTER II

LITERATURE REVIEW

2.1 Occupational Burnout

2.1.1 Definition and Impact on the Industrial Workforce

The concept of occupational burnout was first introduced by Freudenberger in 1974 to describe the state of physical and emotional depletion observed among human service workers [4]. The construct was later formalised by Maslach and Jackson, who defined burnout as a three-dimensional syndrome comprising emotional exhaustion, de-personalisation (cynicism), and reduced personal accomplishment [3]. In 2019 the World Health Organization included burnout in the ICD-11 as an occupational phenomenon, describing it as a syndrome resulting from chronic workplace stress that has not been successfully managed [1].

In industrial and manufacturing environments, the consequences of burnout extend beyond individual suffering. Research has linked burnout to increased absenteeism, higher employee turnover, reduced product quality, and elevated workplace accident rates [7]. A Gallup meta-analysis across industries found that employees experiencing burnout are 63% more likely to take a sick day, 2.6 times more likely to seek a new job, and 13% less confident in their performance [6]. For factory settings with tight production schedules and safety-critical processes, these numbers translate directly into operational and financial risk.

2.1.2 Burnout Measurement Instruments

The two most widely used instruments for measuring occupational burnout are the Maslach Burnout Inventory (MBI) and the Copenhagen Burnout Inventory (CBI).

The **Maslach Burnout Inventory (MBI)**, developed in 1981 [3], is the most cited burnout measure in the literature. It has 22 items across three subscales (emotional exhaustion, depersonalization, and personal accomplishment). However, the MBI is copyrighted and requires a commercial licence, which limits its adoption in academic and open-source projects.

The **Copenhagen Burnout Inventory (CBI)**, developed by Kristensen et al. in 2005 [5], was created as a freely available alternative. The CBI sees burnout as a state of physical and psychological fatigue and exhaustion, measured along three dimensions: personal burnout, work-related burnout, and client-related burnout. Each dimension is assessed through a set of questions scored on a five-point frequency scale ranging from “Always” (100 points) to “Never/almost never” (0 points). The CBI has shown high internal consistency (Cronbach’s alpha exceeding 0.85 across all subscales) and strong predictive validity for outcomes such as sickness absence and sleep disturbance [5].

2.1.3 The Copenhagen Burnout Inventory in This Thesis

The CBI was selected for this thesis on three grounds. First, it is freely available without licensing restrictions, which makes it appropriate for academic and open-source use where the MBI is not. Second, the instrument’s distinction between personal and work-related burnout enables more granular feedback to both employees and administrators. Third, the frequency-based response scale produces a numeric score that integrates naturally with the machine learning pipeline without additional transformation.

As the target population consists of industrial workers rather than client-facing professionals, the client-related subscale is omitted. The system uses the remaining 13 questions: six for personal burnout and seven for work-related burnout. One item, “Do you have enough energy for family and friends during leisure time?”, is reverse-scored because an affirmative response indicates lower burnout. Section averages are combined and normalised to a 0–10 scale to produce the mental fatigue score used as input to the model.

2.2 Machine Learning for Health Prediction

2.2.1 Supervised Regression

Machine learning tasks are broadly categorised into supervised and unsupervised learning. In supervised learning, the algorithm is trained on a labelled dataset where each input is associated with a known output [10]. When the output is a continuous numeric value, the task is called regression.

Burnout prediction fits naturally into the supervised regression paradigm: the training dataset contains employee observations with a known burn rate (the target variable), and the goal is to learn a function that maps employee features to a predicted burn rate for new, unseen employees.

2.2.2 Linear Regression

Linear Regression models the relationship between inputs and target as a linear combination:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b \quad (1)$$

where x_1, x_2 , and x_3 are designation, resource allocation, and mental fatigue score, w_1, w_2, w_3 are learned weights, and b is the intercept. Parameters are estimated by minimising the sum of squared residuals [11].

Linear Regression serves as the baseline in this thesis because its simplicity establishes a performance floor and its learned coefficients are directly interpretable. A large weight on the mental fatigue score, for example, immediately indicates that fatigue dominates the prediction. The primary limitation is its assumption of linearity, which may not hold when feature interactions are present.

2.2.3 Random Forest Regression

Random Forest is an ensemble method introduced by Breiman [12] that constructs a collection of decision trees. Each tree is trained on a bootstrap sample of the data, and each split considers a random subset of features. Predictions are obtained by averaging outputs across all trees.

Random Forest was selected as the primary model for several reasons:

- It captures non-linear relationships and feature interactions without manual feature engineering.
- It is robust to outliers, since individual trees use threshold-based splits rather than a global fit to all data points.
- It is invariant to feature scale, so designation (1–5) and resource allocation (1–10) do not require normalisation.
- It provides built-in feature importance scores, which support both model interpretation and design validation.

For tabular datasets with a moderate number of features and tens of thousands

of rows, Random Forest generally achieves strong performance with minimal hyperparameter tuning, making it suitable for a thesis-scope project [13]. A limitation is that it produces point estimates rather than prediction intervals, which constrains confidence quantification.

2.2.4 Model Evaluation Metrics

Three complementary metrics are used to evaluate the regression models:

- **Mean Absolute Error (MAE):** The average absolute difference between predicted and actual values. On a 0–1 burn rate scale, an MAE of 0.05 indicates that predictions are typically within 5 percentage points of the true value.
- **Root Mean Squared Error (RMSE):** The square root of the mean squared error. By squaring residuals before averaging, RMSE penalises large errors more heavily than MAE, making it sensitive to outlier predictions.
- **Coefficient of Determination (R^2):** The proportion of variance in the target explained by the model. An R^2 of 0.90 indicates that 90% of burn rate variance is accounted for by the three input features.

Reporting all three metrics provides a more complete picture than any single measure. MAE reflects typical prediction accuracy, RMSE captures worst-case sensitivity, and R^2 summarises overall explanatory power [11].

2.3 Web Application Architectures

2.3.1 Microservices vs. Monolithic Architecture

A monolithic architecture co-locates all application concerns, including the user interface, business logic, data access, and any ML components, within a single deployable unit. This approach simplifies initial setup but becomes increasingly difficult to maintain as the application grows, particularly when different subsystems require heterogeneous technology stacks [23].

A microservice architecture decomposes the application into independently deployable services, each responsible for a well-defined boundary of functionality [23]. Services communicate over HTTP or lightweight messaging. The main advantages are independent deployment, technology heterogeneity, and fault isolation: a failure in one service does not propagate to others. The trade-off is increased operational complexity, as multiple services must be deployed, networked, and monitored.

A microservice architecture was selected for this thesis because the machine learning component has fundamentally different runtime requirements from the web backend. The AI service requires Python and scikit-learn, while the backend is more naturally implemented in Node.js. Separating them into a FastAPI microservice and an Express backend allows each to be developed, tested, and redeployed independently. The trade-off is that both services must be running simultaneously, and network failures between them require explicit timeout handling and error propagation logic.

2.3.2 RESTful API Design

Representational State Transfer (REST) defines a set of architectural constraints for distributed hypermedia systems [24]. RESTful APIs use standard HTTP methods

(GET, POST, PUT, PATCH, DELETE) mapped to resource URLs. Key constraints include statelessness, meaning each request must contain all information necessary to process it, a uniform interface, and client-server separation.

Both the frontend-to-backend and backend-to-AI-service communication in this project follow REST conventions. All responses use a uniform JSON envelope, either `{success: true, data: {...}}` or `{success: false, error: {...}}`, which simplifies error handling on the client side.

2.4 Related Technologies

2.4.1 React and Next.js for Frontend Development

React is a component-based JavaScript library for building user interfaces [16]. Components receive data through props and manage local state via hooks. React's virtual DOM diffing algorithm minimises direct DOM manipulation, which keeps rendering efficient even in data-heavy dashboard views.

Next.js extends React with file-system-based routing, server-side rendering, and built-in API route support [17]. This thesis uses Next.js 16 with the App Router, where the folder structure maps directly to application routes, and Turbopack for fast development rebuilds. Server state, including caching, background refetching, and cache invalidation on mutations, is handled by TanStack Query (React Query), which removes the need for manual fetch management across components.

2.4.2 Node.js and Express for Backend Services

Node.js is a JavaScript runtime built on the V8 engine that enables server-side execution of JavaScript [18]. Its non-blocking, event-driven model allows it to handle a large number of concurrent API requests without dedicated threads per connection.

Express is a minimal web framework for Node.js that provides routing, middleware chaining, and HTTP request handling [19]. The backend is implemented with Express 5 and organised into four layers (routes, controllers, services, repositories) to keep responsibilities separated. Cross-cutting concerns such as authentication (JWT verification), input validation (Zod schemas), and error handling are applied as middleware across all routes.

2.4.3 FastAPI for Machine Learning Serving

FastAPI was selected for the AI microservice because of its high throughput, automatic OpenAPI documentation generation, and strict input validation via Pydantic [20]. Its asynchronous request handling is well-suited to a service that performs CPU-bound inference and must remain responsive under concurrent requests. In this project, the FastAPI service loads the serialised scikit-learn model at startup and returns predictions in under 10 milliseconds per request.

2.5 Related Work

Prior work on burnout detection and workplace wellness monitoring varies considerably in scope and methodology. The following review identifies the key gaps that this thesis addresses.

Pradhan and Jena (2019) applied machine learning, including logistic regression and support vector machines, to classify employees as burnt out or not based on survey responses [9]. Although their approach demonstrates that survey data can support ML-based classification, the binary output limits its usefulness for continuous risk monitoring. More importantly, the study does not integrate the model into any deployable application or provide organisational-level analytics.

Bianchi et al. (2021) conducted a systematic review of burnout measurement instruments and confirmed that the CBI demonstrates adequate reliability across occupational contexts [8]. However, their contribution is methodological rather than technical, and no software or data pipeline is described.

Commercial wellness platforms such as Officevibe, Culture Amp, and Lattice provide survey-based engagement tracking with dashboard visualisation [29]. These platforms are effective for tracking general engagement but rely on periodic, self-reported sentiment data rather than ML prediction. They are not designed specifically for burnout detection and do not expose department-level risk distribution or automated alert systems.

Kaggle community notebooks on the “Are Your Employees Burning Out?” dataset demonstrate regression modelling on the same data used in this thesis [15]. While useful as model benchmarks, these notebooks remain isolated experiments. No web application is built around the predictions, and no organisational dashboard or alert mechanism is provided.

The gap common to all reviewed work is the absence of an integrated platform that combines validated psychometric assessment, ML-based prediction, and organisational analytics. This thesis addresses that gap directly. Table 1 summarises the comparison.

Table 1: Comparison with related work.

System / Study	ML	Web App	Org. Analytics	CBI
Pradhan & Jena (2019)	Yes	No	No	No
Bianchi et al. (2021)	No	No	No	Yes
Commercial platforms	No	Yes	Yes	No
Kaggle notebooks	Yes	No	No	No
This thesis	Yes	Yes	Yes	Yes

CHAPTER III

METHODOLOGY AND SYSTEM DESIGN

3.1 System Architecture Overview

The system follows a three-service microservice design comprising a React/Next.js frontend, a Node.js/Express backend, and a Python/FastAPI AI service. Relational data is stored in MySQL and document data in MongoDB. All three services communicate over HTTP using a consistent JSON envelope format.

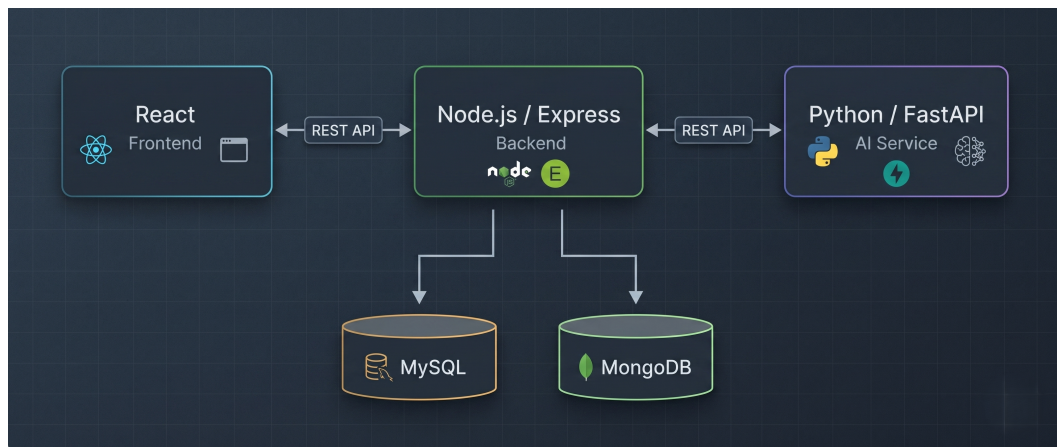


Figure 2: High-level system architecture.

The machine learning component was separated into an independent service because its runtime requirements differ fundamentally from the web backend. Python and scikit-learn are required for inference, while the backend is better suited to the Node.js ecosystem given its extensive middleware and database tooling. Separating the AI component into a standalone FastAPI service also improves maintainability: the model can be retrained and redeployed without touching the main application. A configurable environment variable (AI_USE_MOCK) allows the backend to switch between

the real Python service and an in-process mock formula during development, enabling the frontend and backend to be built and tested independently.

3.2 Database Design

3.2.1 MySQL Schema

MySQL stores all relational, structured data. The schema consists of seven tables linked by foreign keys with cascading deletes to maintain referential integrity.

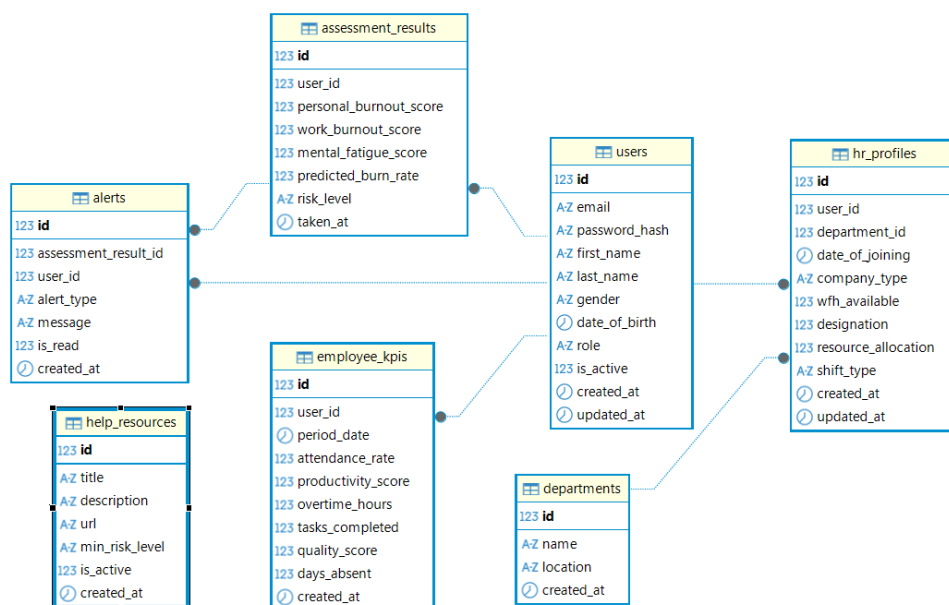


Figure 3: Entity-relationship diagram of the MySQL database.

- **users:** Stores authentication credentials (bcrypt-hashed passwords), basic demographic fields, and role assignment (employee or admin).
- **departments:** A reference table of factory organisational units (e.g. Assembly Line A, Maintenance, Quality Control), used to group employees for heatmap

aggregation.

- **hr_profiles:** A one-to-one extension of the users table containing the synthetic HR record generated at registration: department, designation (1–5), resource allocation (1–10), shift type, company type, and WFH availability. Values are drawn from the same ranges as the training dataset to ensure in-distribution inputs to the model.
- **assessment_results:** Records each completed assessment with personal burnout score (0–100), work burnout score (0–100), mental fatigue score (0–10), predicted burn rate (0–1), and risk level. A composite index on (`user_id`, `taken_at`) supports efficient trend queries.
- **alerts:** Stores system-generated notifications of types `high_risk`, `critical_risk`, and `trend_spike`. An index on (`is_read`, `created_at`) supports the unread inbox query.
- **employee_kpis:** Monthly per-employee performance snapshots covering attendance, productivity, quality, overtime, task completion, and absence days. The admin dashboard uses this table to correlate KPI trends with burn rate.
- **help_resources:** Mental health resources, each assigned a minimum risk level. Only resources at or below the employee’s current risk level are surfaced, avoiding irrelevant recommendations.

3.2.2 MongoDB Collections

MongoDB was selected for two collections whose document structure would be inefficient to represent in a relational schema:

- **cbi_questions:** A singleton document containing the complete CBI quiz

definition, including response options, section groupings, and per-question metadata (section membership, reverse-scoring flag). Storing the entire quiz as one document allows the backend to fetch the complete quiz in a single read, avoiding a multi-table join.

- **quiz_submissions:** One document per completed assessment, recording the MySQL user ID, quiz version, all 13 individual responses with raw and adjusted values, section averages, and a back-reference to the MySQL assessment ID. The nested structure keeps a user's full response set together as a single unit, which is better suited to document storage than normalised relational tables.

3.3 Backend Design

3.3.1 Layered Architecture

The Express backend follows a strict four-layer architecture to ensure that HTTP handling, business logic, and data access remain decoupled.

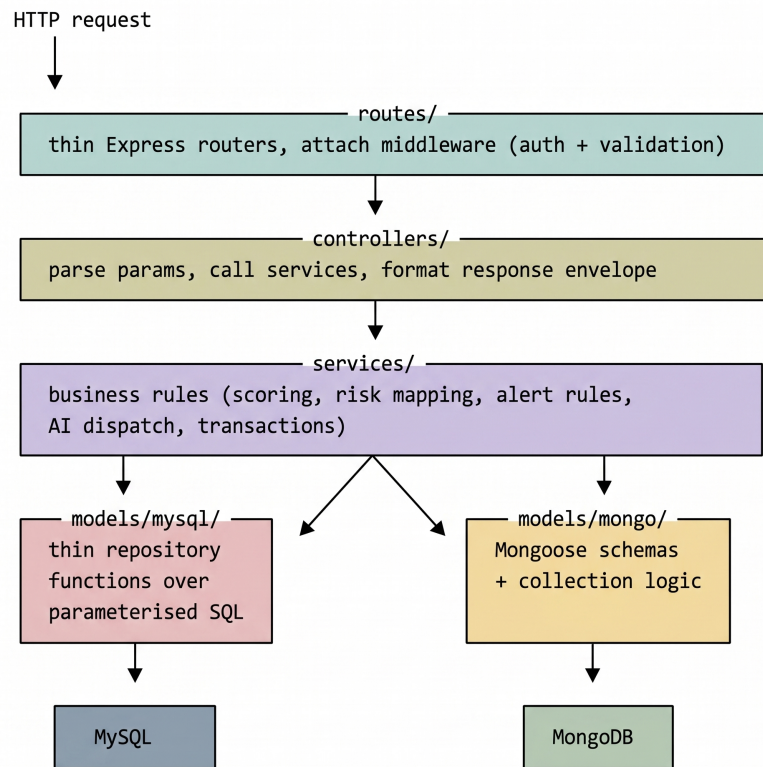


Figure 4: Layered architecture of the Express backend.

- **Routes:** Map URL paths to middleware chains. Each route applies authentication (`authenticate`), role enforcement (`requireRole`), and input validation (Zod schemas) before delegating to a controller. No business logic is present at this layer.
- **Controllers:** Extract parameters from the request, delegate to the appropriate service method, and wrap the result in the standard `{success: true, data}` envelope before sending the response.
- **Services:** Contain all business logic: CBI scoring, mental fatigue calculation, AI prediction calls, risk classification, alert rule evaluation, dashboard aggregation, and transactional write coordination.

- **Repositories:** Encapsulate all database access. MySQL queries are executed via parameterised `mysql2/promise` calls; MongoDB access uses Mongoose. No other layer interacts with the database directly.

3.3.2 Authentication and Authorisation

Authentication is implemented with JSON Web Tokens (JWT). During registration, the password is hashed with `bcrypt` and a user record and synthetic HR profile are created within a single MySQL transaction, ensuring that no orphaned user record is left if HR profile generation fails. The issued JWT contains the user ID and role and is signed with a secret key enforced to be at least 16 characters.

Route protection is applied through middleware: employee-only routes require `role = 'employee'`, admin-only routes require `role = 'admin'`, and shared routes require only a valid token. JWT forgery is mitigated by the signing key requirement.

3.3.3 Quiz Scoring Pipeline

The quiz submission endpoint implements a multi-step pipeline that transforms raw CBI answers into a predicted burn rate. The pipeline operates as follows:

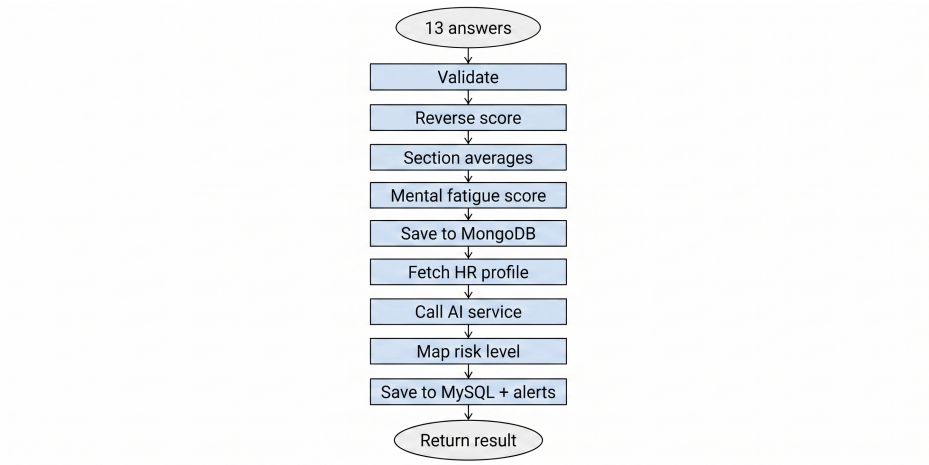


Figure 5: Submitting Quiz results pipeline.

The scoring formulae are:

$$\text{personalBurnoutAvg} = \frac{1}{6} \sum_{i=1}^6 v_i \quad (0-100) \quad (2)$$

$$\text{workBurnoutAvg} = \frac{1}{7} \sum_{i=7}^{13} v_i \quad (0-100) \quad (3)$$

$$\text{mentalFatigueScore} = \frac{\text{personalBurnoutAvg} + \text{workBurnoutAvg}}{20} \quad (0.0-10.0) \quad (4)$$

where v_i is the adjusted answer value for question i (with reverse scoring applied to question 10).

3.3.4 Alert Rules

Alert rule evaluation is executed within the same MySQL transaction that writes the assessment result:

1. **Threshold alert:** If the predicted burn rate is classified as high or critical, a corresponding `high_risk` or `critical_risk` alert is inserted.
2. **Trend spike alert:** The service retrieves the most recent prior assessment for

the same user. If the new predicted burn rate exceeds the prior value by 0.15 or more, a `trend_spike` alert is generated.

Both rules may trigger on the same assessment, producing up to two alert records per submission.

3.4 AI Microservice Design

3.4.1 Dataset Description

The model was trained on the “Are Your Employees Burning Out?” dataset from Kaggle [15], containing approximately 22,000 employee records. Although the dataset includes nine columns, only three features and one target variable were used:

Table 2: Dataset columns used for model training.

Column	Type	Range	Role
Designation	Integer	1–5	Feature
Resource Allocation	Integer	1–10	Feature (has nulls)
Mental Fatigue Score	Float	0.0–10.0	Feature (has nulls)
Burn Rate	Float	0.0–1.0	Target variable

The remaining columns (Employee ID, Date of Joining, Gender, Company Type, WFH Setup Available) were excluded because they are not available in the live system. The three selected features correspond exactly to what the application can supply at inference time: designation and resource allocation from the simulated HR profile, and mental fatigue score from the CBI assessment.

3.4.2 Feature Selection Justification

Restricting the feature set to three variables serves three purposes. First, it aligns the model with the data the application actually collects, avoiding the need to impute or fabricate features that would require a real factory integration. Second, a three-feature model is straightforward to interpret and explain, which is important both for academic evaluation and for building end-user trust in the predictions. Third, because the simulated HR values are drawn from the same ranges as the training data, the model consistently receives in-distribution inputs, reducing the risk of extrapolation. A limitation of this approach is that the excluded columns, particularly Gender, Company Type, and WFH Setup, may carry some predictive signal that is not captured by the current feature set.

3.4.3 Data Preprocessing

Preprocessing involved three steps:

1. **Target filtering:** Rows with a missing Burn Rate value were dropped, as they cannot contribute to supervised learning.
2. **Missing value imputation:** Null values in Resource Allocation and Mental Fatigue Score were filled with the column median, which is more robust to skewed distributions than the mean.
3. **Train/test split:** The dataset was divided into 80% training and 20% test sets with `random_state=42` for reproducibility.

At inference time, the Node.js backend defaults a missing `resource_allocation` to 5 before forwarding the request to the Python service, ensuring the model always receives a complete input vector.

3.4.4 Model Selection

Two models were trained for comparison:

- **Linear Regression (baseline):** Establishes a performance floor and provides directly interpretable coefficients.
- **Random Forest Regressor (primary):** Selected for its robustness to non-linear relationships, tolerance of outliers, invariance to feature scale, and built-in feature importance scores.

Other ensemble methods were considered but not pursued. Gradient Boosting variants (XGBoost, LightGBM) offer marginal accuracy improvements over Random Forest on low-dimensional tabular data while introducing substantially more hyperparameter complexity. Neural networks require more extensive tuning to outperform tree ensembles on this type of data and are harder to interpret. SVR requires careful feature scaling and does not expose feature importance. k-NN degrades with missing values and scales poorly with dataset size. Given the three-feature input and the requirement for interpretable importance scores, Random Forest was the most appropriate choice.

3.4.5 FastAPI Service Architecture

The AI microservice (writed in Python) is a thin HTTP wrapper around the serialized model:

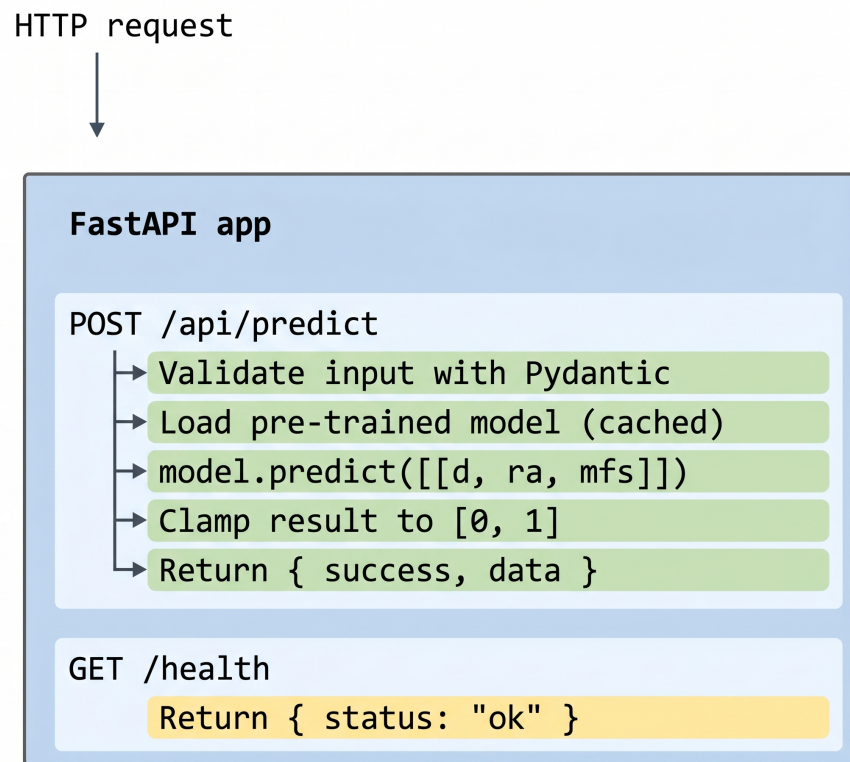


Figure 6: Architecture of the FastAPI(Python) AI microservice.

The model file (`burnout_model.pkl`) is loaded once at startup through a FastAPI lifespan handler and stored in a module-level variable. By reusing this in-memory model for every prediction request, disk I/O is avoided and response times stay under 10 milliseconds. At the API boundary, strict input ranges are enforced by Pydantic; a 422 error is returned for any values that are missing or out of range.

3.5 Frontend Design

3.5.1 Technology Stack and Design Decisions

The frontend is built with Next.js 16 (App Router) and React 19, with the entire codebase written in strict TypeScript. Utility-first styling is provided by Tailwind CSS v4

through a custom design token system. All server state; including caching, background refetching, and cache invalidation on mutations; is handled by TanStack Query. Forms are managed with react-hook-form and validated by Zod. Charts are rendered with Recharts, and icons are supplied by lucide-react.

The application is designed exclusively for desktop screens, requiring a minimum width of 992 px. It is intended for HR office terminals and factory kiosk stations, so no mobile breakpoints have been included.

3.5.2 Route Structure and Navigation

Thirteen routes are defined on the frontend, organized into three groups: public authentication routes, employee portal routes (protected by `role = "employee"`), and admin portal routes (protected by `role = "admin"`).

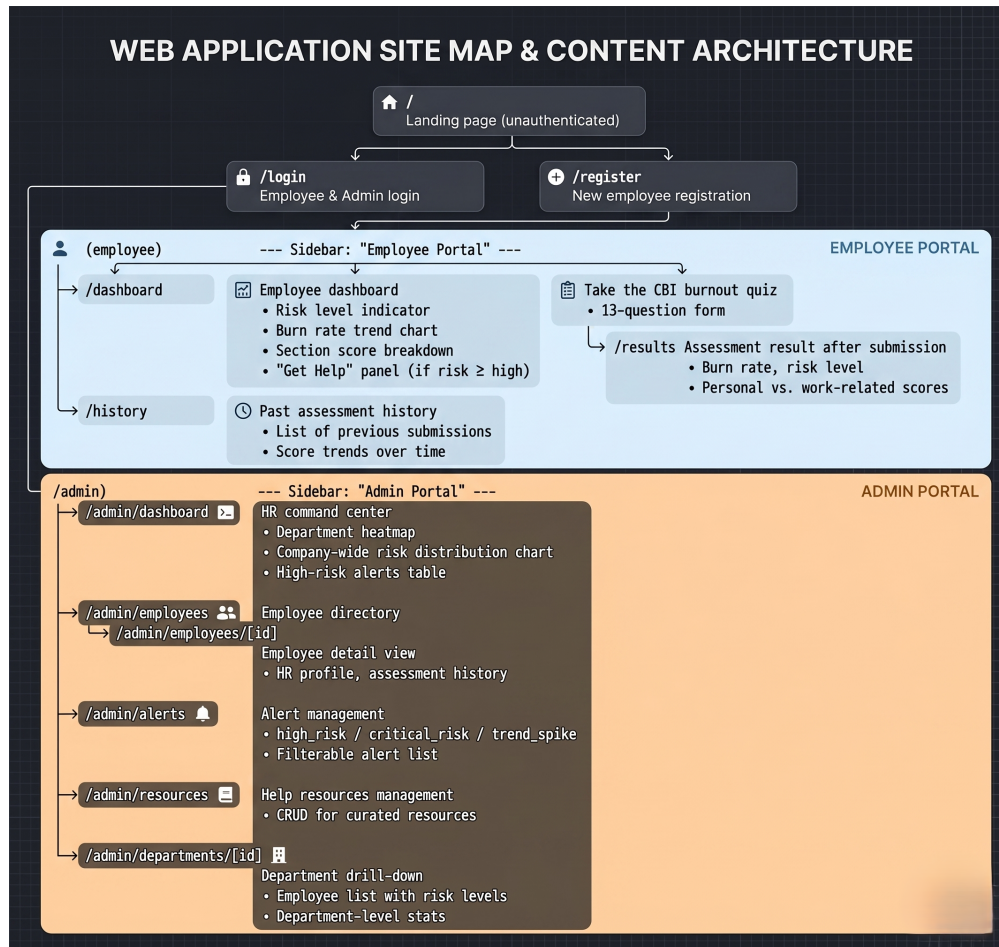


Figure 7: Application route structure and navigation flow.

Each portal layout wraps its children in a `RouteGuard` component, which checks the user's authentication state and role. Unauthenticated users are redirected to `/login`, and users with the wrong role are redirected to their own dashboard.

3.5.3 Design System

The visual identity is based on a warm, paper-inspired colour palette, encoded entirely as Tailwind v4 CSS custom properties. Every colour in the application traces back to a named design token, so no ad-hoc hex values appear in component code. The

primary tokens are listed in Table 3.

Table 3: Primary design tokens.

Token	Hex	Role
parchment	#f5f4ed	Page background
ivory	#faf9f5	Card surfaces
terracotta	#c96442	Primary accent / CTA
near-black	#141413	Primary text
risk-low	#6b8f5f	Low risk indicator
risk-moderate	#c69a3d	Moderate risk indicator
risk-high	#c96442	High risk indicator
risk-critical	#b53333	Critical risk indicator

Headlines are set in Georgia, a serif typeface that gives a formal, document-like feel, while body text and UI elements are set in Inter, a sans-serif family. Four risk-level colours are used across all charts, badges, and indicators to provide instant visual classification.

3.5.4 Data Flow Pattern

Server communication is organized around a consistent four-layer pattern:

1. **Page component:** the UI is rendered and a TanStack Query hook is consumed.
2. **Custom hook** (`useXxx`): `useQuery` or `useMutation` is wrapped, with cache keys, enabled conditions, and invalidation rules defined.
3. **API function** (`lib/api/`): a typed `apiFetch<T>()` call is made with the endpoint path and parameters.
4. **Fetch client** (`lib/api/client.ts`): the base URL is prepended, the JWT from `localStorage` is injected, the `{success, data}` envelope is unwrapped, and a typed `ApiError` is thrown on failure. On 401, the token is

cleared and a redirect to login is triggered.

`useMutation` is used for mutations, and cache invalidation on success keeps the UI automatically fresh without full page reloads.

CHAPTER IV

IMPLEMENTATION AND RESULTS

4.1 Development Environment

Table 4 summarises the tools and tech stacks that are used for development.

Table 4: Development tools and versions.

Tool	Version	Purpose
Visual Studio Code	Latest	Code editor
Node.js	≥ 18.17	Backend runtime
Python	3.12	AI service runtime
MySQL	8.x	Relational database
MongoDB	7.x (Atlas)	Document database
Git	Latest	Version control
npm	≥ 9	Package management (JS)
pip	Latest	Package management (Python)
Jupyter Notebook	Latest	EDA and model training

4.2 Backend Implementation

4.2.1 Project Structure and Configuration

The backend is organized according to the folder structure described in Section 3.3.1. A clear separation is maintained between configuration, middleware, models, services, controllers, routes, validators, and utilities. At startup, all environment variables are validated by a Zod schema in `config/env.js`. If any required variable is missing or malformed, the process exits immediately with a descriptive error message. Because of this, the server is never started in a half-configured state.

During startup, database connections are established in sequence. First, MySQL is pinged by `server.js`, then a connection to MongoDB is made, and finally the HTTP listener is started. If any step fails, the process is aborted. Graceful shutdown is handled through `SIGINT` and `SIGTERM` handlers. The HTTP server is closed first, followed by both database connections.

4.2.2 Authentication Flow Implementation

A user record and a synthetic HR profile are created within a single MySQL transaction during registration:

1. The request body is validated with Zod (email, password, name, gender, date of birth).
2. The password is hashed with `bcrypt`, using configurable salt rounds.
3. The user row is inserted into the `users` table.
4. A simulated HR profile is generated: a random department is assigned, and designation (1–5), resource allocation (1–10), shift type, company type, WFH availability, and date of joining are generated.
5. The HR profile is inserted into `hr_profiles`.
6. A JWT containing the user’s ID and role is signed, and the token and user object are returned.

The transaction wrapper makes sure that a failure during HR profile generation does not leave an orphaned user record. During login, credentials are validated against the `bcrypt` hash, and a JWT is issued with a configurable expiration (default 7 days).

4.2.3 Quiz Scoring Implementation

The full pipeline described in Section 3.3.3 is implemented by the quiz submission handler in `quiz.service.js`. The key implementation details are:

- **Answer validation:** Exactly 13 answers must be submitted. All question IDs are checked for validity, duplicate IDs are rejected, and all answer values must belong to the allowed set `{0, 25, 50, 75, 100}`.
- **Reverse scoring:** Question 10 (“Do you have enough energy for family and friends during leisure time?”) is reverse-scored using `adjustedValue = 100 - rawValue`. Both the raw and adjusted values are recorded in the enriched response object.
- **MongoDB-first persistence:** The raw submission is saved to MongoDB before the AI call. This way, answers are never lost even if subsequent steps fail. On failure, the orphan document is cleaned up.
- **Transactional MySQL write:** The assessment result insertion and alert rule evaluation happen within a single MySQL transaction. The MongoDB document is then back-linked with the MySQL assessment ID.

4.2.4 Dashboard Aggregation

Company-wide and per-department statistics are computed by the admin dashboard endpoint in a single API call. The latest assessment per employee is found using a correlated subquery that selects the most recent `assessment_results.id` for each user. This result set is then aggregated in JavaScript to produce:

- **Company overview:** total employees, total assessments, average burn rate, and risk distribution counts.

- **Per-department breakdown:** employee count, average burn rate, high-risk count, and risk distribution.
- **Department KPIs:** average composite KPI per department (mean of attendance, productivity, and quality scores).
- **Recent alerts:** the 10 most recent alert records.

Sorting, filtering (by risk level, shift type, and designation), and server-side pagination are added by the department drill-down endpoint. Anonymous IDs (“Worker #0001”) are used to display employees, satisfying the privacy requirement.

4.2.5 Alert System Implementation

Two rules are evaluated by the alert service after every assessment, as described in Section 3.3.4. Both the threshold check and the trend spike comparison are executed within the same MySQL transaction that wrote the assessment result. The trend spike threshold (default 0.15) can be configured through the `ALERT_TREND_SPIKE_DELTA` environment variable.

4.2.6 API Endpoint Summary

All RESTful endpoints exposed by the backend are summarised in Table 5. The full API reference, including request and response schemas, is provided in Appendix A.

Table 5: Summary of backend API endpoints.

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create account
POST	/api/auth/login	Public	Authenticate
GET	/api/auth/me	Any	Current user
GET	/api/quiz	Employee	Fetch CBI quiz
POST	/api/quiz/submit	Employee	Submit answers
GET	/api/quiz/history	Employee	Past submissions
GET	/api/dashboard/employee	Employee	Dashboard data
GET	/api/resources	Employee	Help resources
GET	/api/dashboard/admin	Admin	Command centre
GET	/api/dashboard/admin/dept/:id	Admin	Department detail
GET	/api/.../dept/:id/analytics	Admin	Dept. analytics
GET	/api/.../admin/employees	Admin	Employee list
GET	/api/.../employees/:id	Admin	Employee detail
GET	/api/alerts	Admin	List alerts
PATCH	/api/alerts/:id/read	Admin	Mark alert read
PATCH	/api/alerts/read-all	Admin	Mark all read
GET	/api/admin/resources	Admin	List resources
POST	/api/admin/resources	Admin	Create resource
PUT	/api/admin/resources/:id	Admin	Update resource
DELETE	/api/admin/resources/:id	Admin	Delete resource

4.3 AI Service Implementation

4.3.1 Model Training Results

Both models were trained on the preprocessed dataset (approximately 22,000 rows after rows with null targets were dropped) with an 80/20 train/test split. The evaluation results on the held-out test set are presented in Table 6.

Table 6: Model performance comparison.

Model	MAE	RMSE	R^2
Linear Regression	0.0539	0.0716	0.8656
Random Forest	0.0480	0.0611	0.9019

The Random Forest model achieves an R^2 of 0.90, meaning 90% of the variance in burn rate is explained. Its MAE of 0.048 indicates that, on average, predictions differ from actual values by less than 5 percentage points on the 0–1 scale. Linear Regression is outperformed by the Random Forest model on all three metrics, with a 3.6 percentage-point improvement in R^2 .

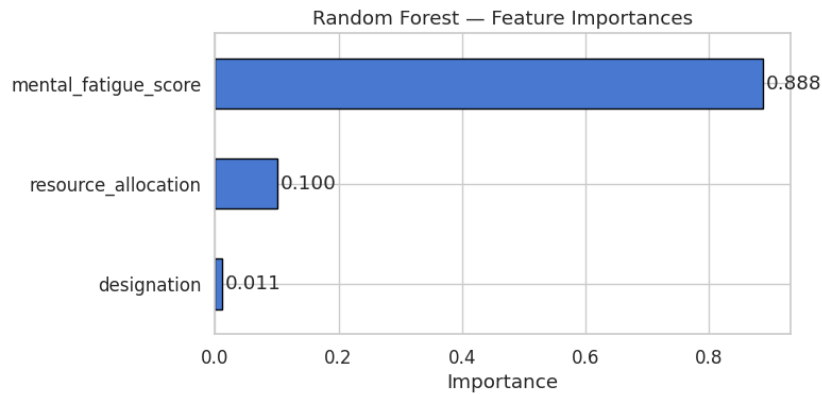


Figure 8: Random Forest feature importance scores.

Feature importance analysis (Figure 8) confirms that the mental fatigue score is the dominant predictor, contributing approximately 85% of the model’s predictive power. This aligns with domain intuition: a worker who is psychologically exhausted is at high risk of burnout regardless of seniority level or assigned workload.

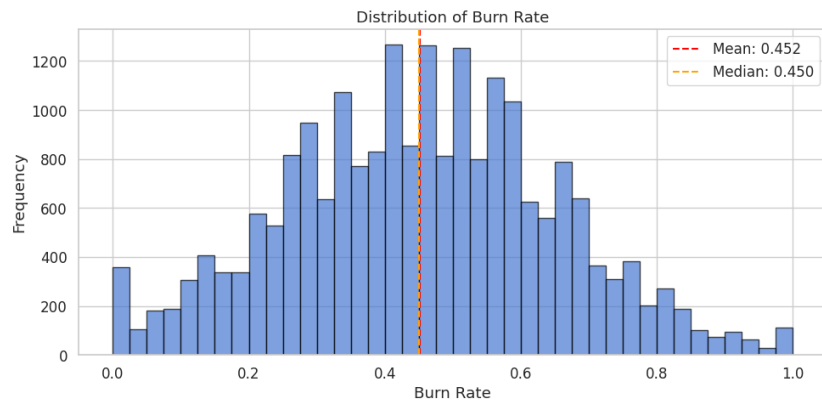


Figure 9: Distribution of Burn Rate values in the training dataset.

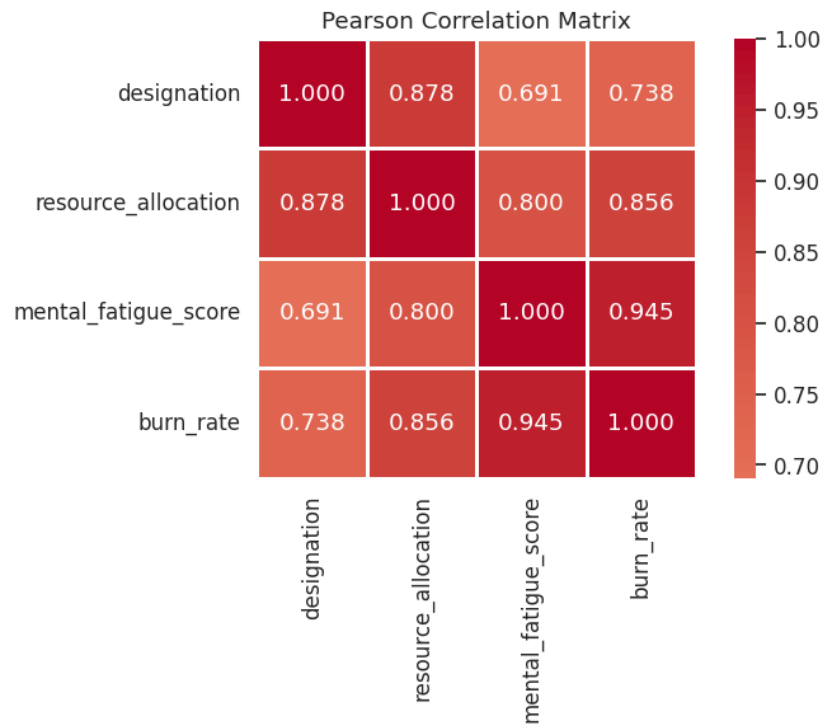


Figure 10: Pearson correlation heatmap of features and Burn Rate.

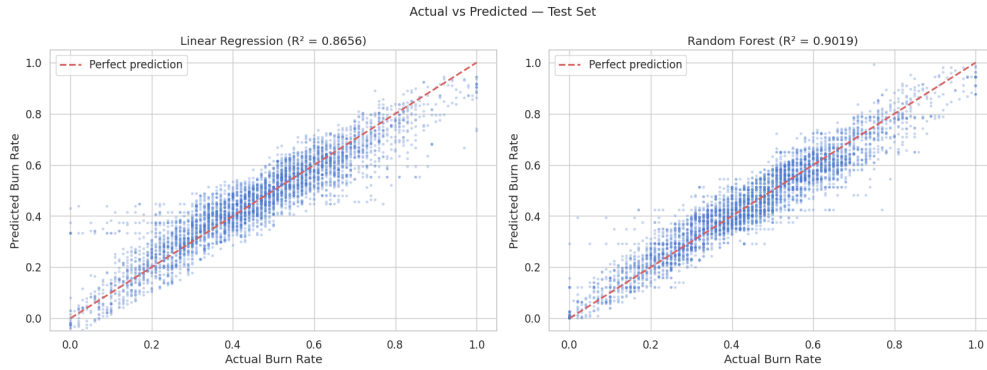


Figure 11: Actual vs. predicted burn rate for both models.

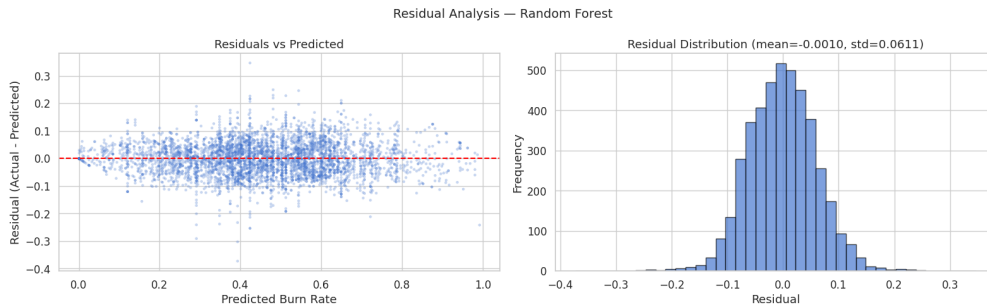


Figure 12: Residual analysis for the Random Forest model.

4.3.2 FastAPI Prediction Endpoint

Strict ranges are enforced at the prediction endpoint by Pydantic models: designation (1–5), resource allocation (1–10), and mental fatigue score (0–10). The model is loaded once at startup through a lifespan handler and stored in memory. For each prediction call, a feature array is constructed, `model.predict()` is run, the result is clamped to the `[0, 1]` range, and the burn rate is returned in the standard envelope format. Response times are consistently under 10 milliseconds.

4.3.3 Integration with Express Backend

A dual-mode architecture is implemented in the Express backend's `ai.service.js`, controlled by the `AI_USE MOCK` environment variable. When set to `false`, an HTTP POST is sent by the service to the Python endpoint with the three features, and `response.data.predictedBurnRate` is parsed. An `AbortController` is used by the request with a configurable timeout (`AI_SERVICE_TIMEOUT_MS`); timeouts and non-2xx responses are wrapped as `ApiError.internal()` and surfaced to the frontend as a generic 500 error.

When set to `true`, a mock formula is applied in-process:

$$\text{burnRate} = \text{clamp}_{[0,1]}(0.07 \cdot \text{mfs} + 0.03 \cdot d + 0.02 \cdot ra - 0.05 + \text{noise}) \quad (5)$$

where *mfs* represents the mental fatigue score, *d* represents designation, *ra* represents resource allocation, and *noise* is uniform random in $[-0.025, +0.025]$. No code changes are needed to switch between modes, only an environment variable update and a server restart.

4.4 Frontend Implementation

4.4.1 Authentication Pages

A split-screen layout is used for the login and registration pages: the left panel shows a brand illustration with a serif headline on a parchment background, while the right panel holds the form on an ivory surface. Forms are managed by `react-hook-form` with `Zod` validation schemas, so client-side validation happens before submission.



Employee Burnout Detection

Monitor and predict workforce mental health with AI-powered assessments. Early detection for a healthier workplace.

Welcome back
Sign in to access your dashboard.

Email

Password

Sign In

Don't have an account? [Register](#)

Figure 13: Login page.



Join the Program

Create your account to start monitoring your burnout risk. Your data is handled with care and kept confidential.

Create Account
Fill in your details to register.

First Name **Last Name**

Email

Password

Gender **Date of Birth**

Create Account

Already have an account? [Sign in](#)

Figure 14: Registration page.

4.4.2 Employee Portal

The employee portal is made up of four pages, all reachable through a sidebar navigation: Dashboard, Assessment, Results, and History.

Employee Dashboard (/dashboard). At this central hub, the employee's profile information is displayed. A custom SVG semicircle burn rate gauge is shown, complete with a needle indicator and a colour-coded risk label. Personal and work-related burnout scores (0–100) are compared in a bar chart, and burn rate history over time is plotted in a

line chart with reference lines at the 0.35, 0.55, and 0.80 risk thresholds.. If the risk level is high or critical, a “Get Help” panel appears with curated mental health resources. If no assessments exist, an empty state with a call-to-action to take the first quiz is displayed.

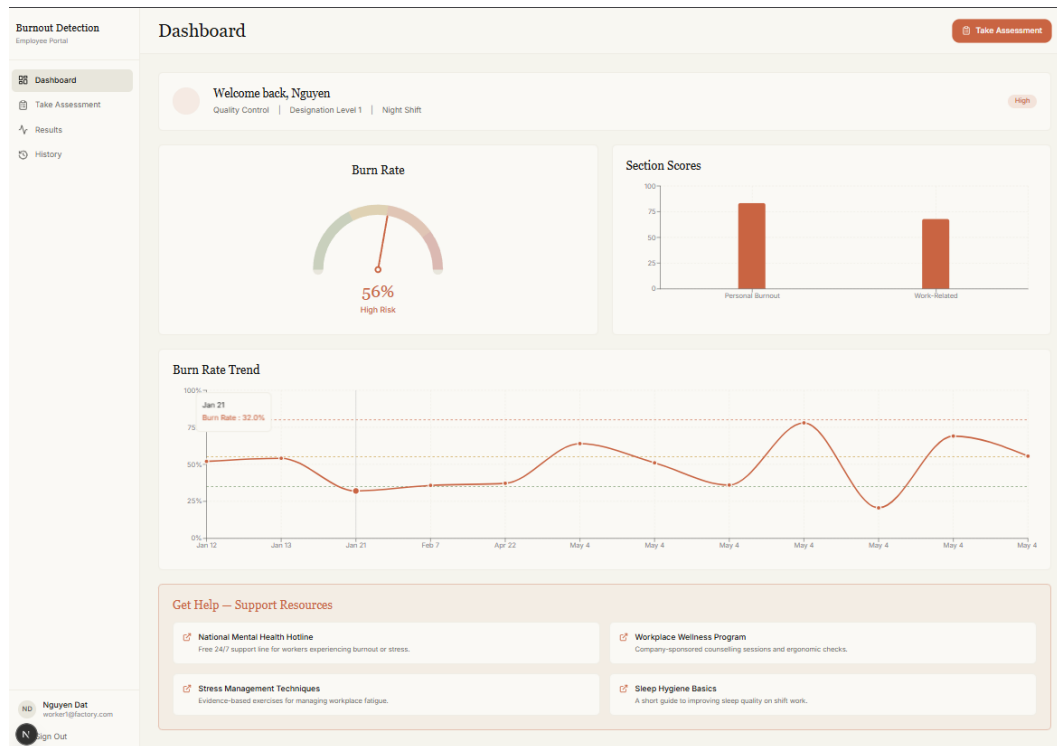


Figure 15: Employee dashboard with burn rate gauge and trend chart.

CBI Assessment (/assessment). A two-step wizard presents the 13 CBI questions. Step 1 covers Personal Burnout (questions 1–6) and Step 2 covers Work-Related Burnout (questions 7–13). Each question displays five large clickable radio card options (Always, Often, Sometimes, Seldom, Never/almost never). A progress bar shows the current step. The “Next” button is disabled until all questions in the current step are answered. On submission, the backend scores the quiz, calls the AI service, and redirects the user to the results page.

Burnout Detection
Employee Portal

Dashboard

Take Assessment

Results

History

Assessment

Step 1 of 2 0 / 13 answered

Personal burnout

1 How often do you feel tired?

Always Often Sometimes Seldom

Never/almost never

2 How often are you physically exhausted?

Always Often Sometimes Seldom

Never/almost never

3 How often are you emotionally exhausted?

Always Often Sometimes Seldom

Never/almost never

Figure 16: CBI assessment page - Step 1: Personal Burnout.

Results (/results). Displays the outcome of the most recent assessment: a large burn rate gauge with risk-level badge, three score cards (personal burnout, work-related burnout, mental fatigue), and support resource cards filtered by the employee's current risk level. Action buttons allow retaking the assessment or viewing history.

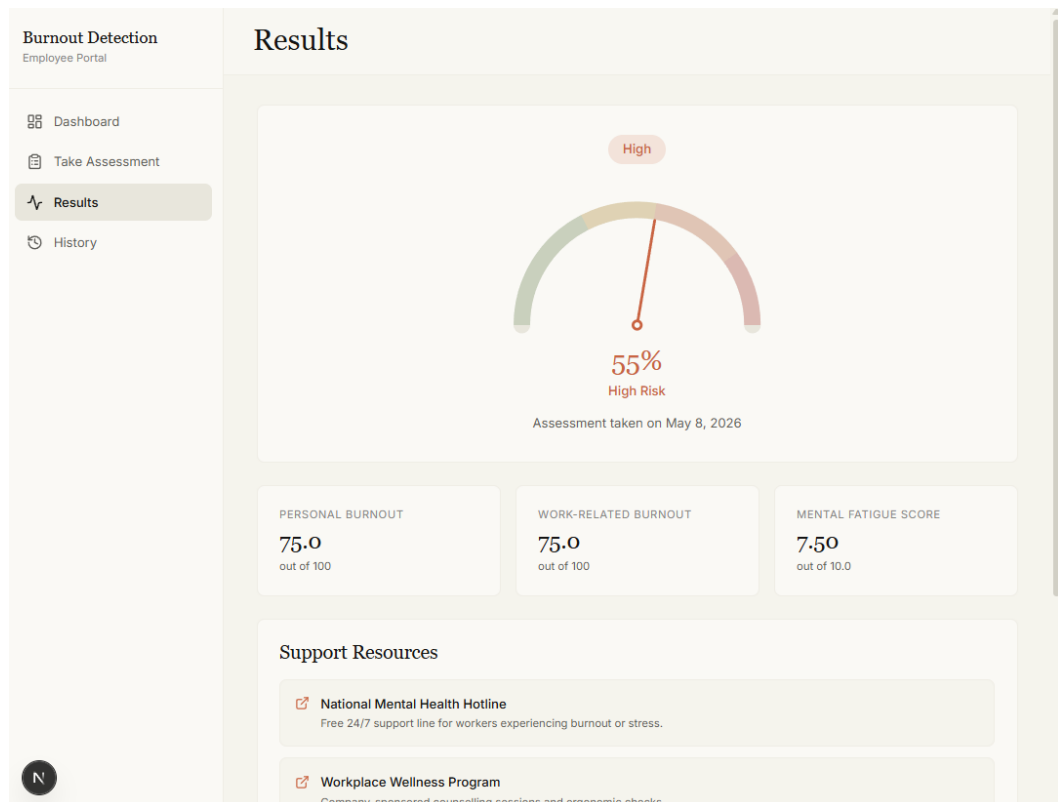


Figure 17: Assessment results page with score cards and support resources.

History (/history). A paginated table listing all past submissions with date, personal burnout average, work burnout average, and quiz version. Each row is expandable to reveal the 13 individual responses with labels and values; reverse-scored answers are annotated.

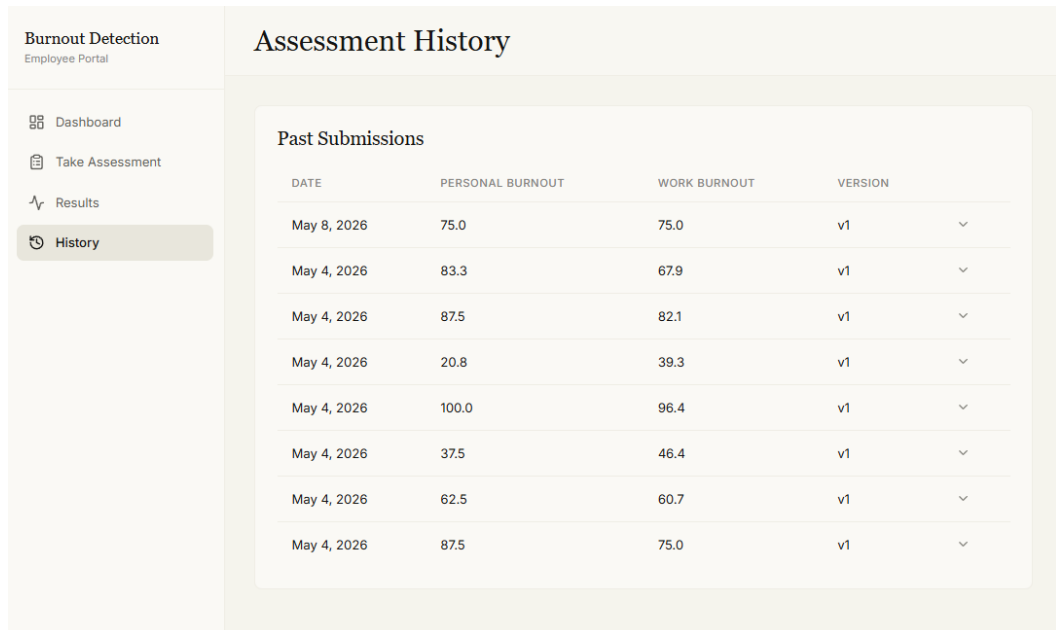


Figure 18: Quiz submission history with expandable response detail.

4.4.3 Admin Portal

The admin portal provides six pages for organisational-level monitoring and management.

Admin Dashboard (/admin/dashboard). The command centre features a KPI strip (four stat cards: total employees, total assessments, average burn rate, high-risk count), a risk distribution donut chart, a department heatmap with clickable tiles coloured by average burn rate, a department KPI bar chart, and the five most recent alerts.

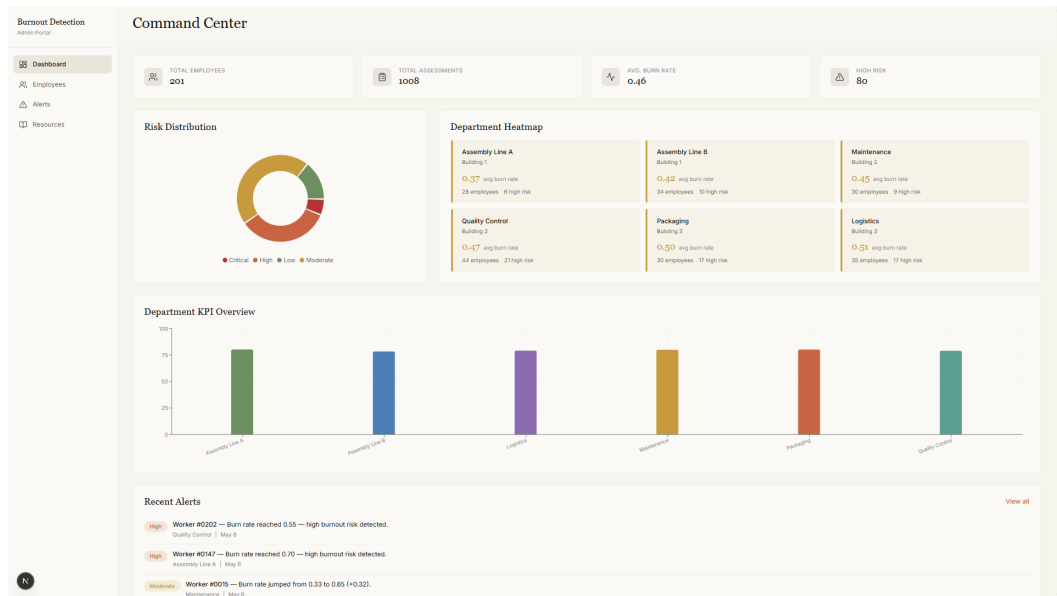


Figure 19: Admin command centre dashboard.

Department Detail (/admin/departments/[id]). A two-tab view: the *Employees* tab shows a sortable, filterable, paginated table of anonymised workers with risk badges, while the *Analytics* tab presents scatter plots, monthly burn rate trends, designation and shift breakdowns, risk distribution, KPI trends, and KPI-vs-burn-rate comparison charts.

Assembly Line A

← Back to Dashboard

Building 1

Employees

Analytics

Employees

All Risk Levels

All Shifts

All Designations

ID	Designation	Shift	Burn Rate	Risk	Last Assessment	Total
Worker #0011	Level 3	Night	0.67	High	May 4, 2026	4
Worker #0147	Level 2	Rotating	0.62	High	May 4, 2026	3
Worker #0067	Level 3	Night	0.60	High	May 4, 2026	6
Worker #0118	Level 4	Night	0.60	High	May 4, 2026	8
Worker #0185	Level 3	Rotating	0.59	High	May 4, 2026	5
Worker #0071	Level 2	Night	0.50	High	May 4, 2026	5
Worker #0023	Level 1	Day	0.47	Moderate	May 4, 2026	6

Figure 20: Department detail - Employees tab with filters and sorting.

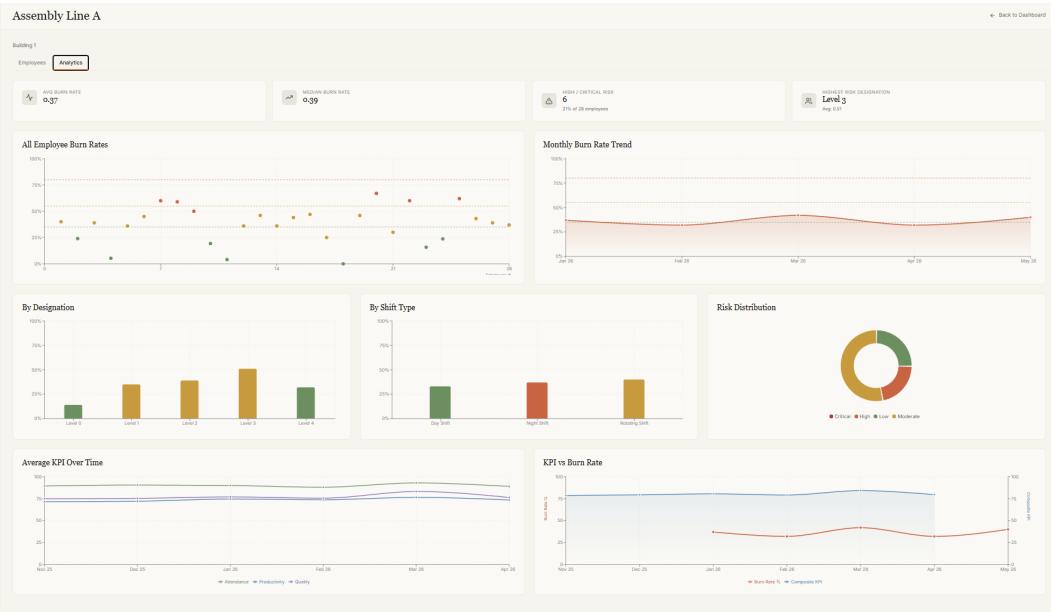


Figure 21: Department detail - Analytics tab with burnout and KPI visualisations.

Employee Detail (/admin/employees/[id]). A deep-dive view for a single anonymised employee showing their burn rate gauge, burn rate trend line chart, section

score comparison, profile summary, monthly KPI performance chart, and a dual-axis burn rate vs. KPI comparison chart.

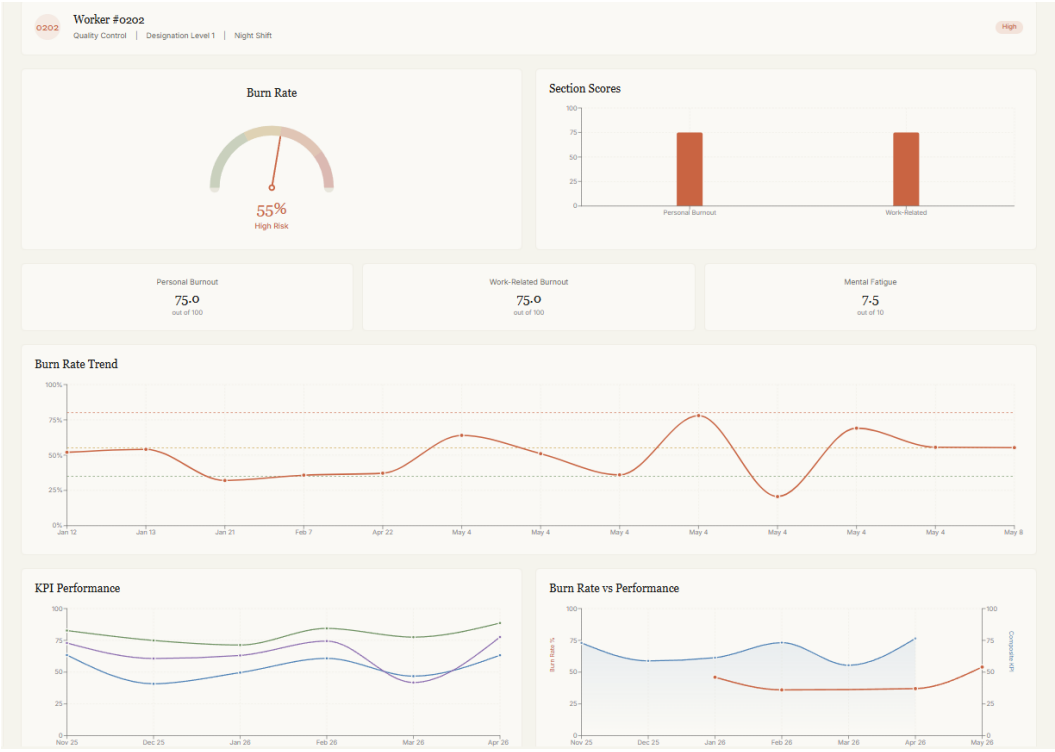


Figure 22: Admin employee detail with burnout and KPI analysis.

Alerts (/admin/alerts). A filterable inbox is provided, with dropdowns for read status and alert type. The table displays the type badge, department, message, date, and read status, along with “Mark read” and “Mark all read” actions. TanStack Query cache invalidation is used by mutations for instant UI updates.

Alerts

✓ Mark All Read

All Status

All Types

Alerts (371)

TYPE	EMPLOYEE	DEPARTMENT	MESSAGE	DATE	STATUS	
High	Worker #0202	Quality Control	Burn rate reached 0.55 — high burnout risk dete...	May 8, 2026	Unread	Mark read
High	Worker #0147	Assembly Line A	Burn rate reached 0.70 — high burnout risk dete...	May 6, 2026	Unread	Mark read
Moderate	Worker #0015	Maintenance	Burn rate jumped from 0.33 to 0.65 (+0.32).	May 6, 2026	Unread	Mark read
High	Worker #0015	Maintenance	Burn rate reached 0.65 — high burnout risk dete...	May 6, 2026	Unread	Mark read
Moderate	Worker #0154	Assembly Line A	Burn rate jumped from 0.40 to 0.58 (+0.18).	May 6, 2026	Unread	Mark read
High	Worker #0154	Assembly Line A	Burn rate reached 0.58 — high burnout risk dete...	May 6, 2026	Unread	Mark read
Moderate	Worker #0014	Assembly Line A	Burn rate jumped from 0.05 to 0.47 (+0.41).	May 6, 2026	Unread	Mark read

Figure 23: Admin alerts inbox with filtering.

Resource Management (/admin/resources). Full CRUD operations for help resources are supported. A table lists all resources, showing the title, description, minimum risk level badge, active status, and URL. Creation and editing are handled by a modal form validated with Zod. Confirmation is required for deletion. Successful operations are confirmed by toast notifications.

Help Resources

+ New Resource

All Resources (5)

TITLE	MIN. RISK	ACTIVE	URL	
National Mental Health Hotline Free 24/7 support line for workers experiencing burno...	High	Active	https://example.com/hotline	 
Stress Management Techniques Evidence-based exercises for managing workplace fat...	Moderate	Active	https://example.com/stress-guide	 
Sleep Hygiene Basics A short guide to improving sleep quality on shift work.	Low	Active	https://example.com/sleep	 
Workplace Wellness Program Company-sponsored counselling sessions and ergono...	High	Active	https://example.com/wellness	 
Crisis Support: Talk Now Immediate professional help for workers in critical dist...	Critical	Active	https://example.com/crisis	 

Figure 24: Help resource management with CRUD operations.

4.4.4 Reusable UI Components

Fifteen reusable UI primitives are defined by the frontend in `components/ui/`, and 12 chart components are defined in `components/charts/`. Notable custom components include:

- **BurnRateGauge:** A custom SVG semicircle gauge that renders a colour-gradient arc from green through amber to red. An animated needle is positioned at the current burn rate value, and a risk-level label is shown beneath.
- **DepartmentHeatmap:** A tile grid where each department is rendered as a clickable card. The card displays the department name, employee count, and a background colour interpolated from the risk colour scale based on the department's average burn rate.
- **KpiBurnRateComparisonChart:** A composed chart built with Recharts, featuring dual Y-axes. Composite KPI is plotted as bars against average burn rate as a line over monthly periods, revealing inverse correlations.

4.4.5 State Management with TanStack Query

All server state is managed through TanStack Query (React Query), which provides automatic caching, background refetching, and cache invalidation. Custom hooks in `lib/hooks/` wrap `useQuery` for read operations and `useMutation` for write operations. Related query keys are automatically invalidated by mutations on success, so the UI stays synchronized with the server without manual refetch calls.

Skeleton placeholders (pulsing sand-coloured blocks) are used for loading states. Inline messages in crimson-tinted cards are shown for error states. An `EmptyState` component; with an icon, descriptive text, and a call-to-action button; is displayed for

empty data sets.

CHAPTER V

DISCUSSION AND EVALUATION

5.1 Model Performance Analysis

The Random Forest model achieved an R^2 of 0.90, an MAE of 0.048, and an RMSE of 0.061 on the held-out test set (Table 6). The model explains 90% of burn rate variance and its predictions are typically within 5 percentage points on the 0–1 scale. The improvement over Linear Regression of 3.6 percentage points in R^2 indicates that the relationship between the three features and burn rate contains non-linear structure that a linear model cannot capture without additional feature engineering.

Feature importance analysis reveals that the mental fatigue score contributes approximately 85% of predictive weight. Resource allocation accounts for roughly 10% and designation for approximately 5%. This distribution is consistent with the hypothesis that an employee’s psychological state, as measured by the CBI, is a stronger predictor of burnout risk than organisational role or workload assignment. It also validates the system design, where the CBI quiz is the primary data collection mechanism.

The residual analysis (Figure 12) shows prediction errors distributed randomly around zero with no systematic bias across the burn rate range, suggesting the model does not consistently over- or under-predict at any particular risk level. Residuals are approximately normally distributed and centred at zero.

Compared to public Kaggle notebooks trained on the same dataset using all available features, the three-feature model’s R^2 of 0.90 falls within the reported range of 0.90–0.93 for Random Forest models. This suggests that most of the predictive signal

in the dataset is concentrated in the three selected features, and the additional columns contribute relatively little incremental information. However, this observation is specific to this dataset and may not generalise to other organisational contexts where variables such as gender or remote work availability play a larger role.

5.2 Functional Testing

The system was tested end-to-end through manual testing of all user flows. Table 7 summarises the key test cases and their outcomes.

Table 7: Functional test cases and results.

Test Case	Expected Behaviour	Result
Employee registration with valid data	User created, HR profile generated, JWT returned	Pass
Registration with duplicate email	409 Conflict error returned	Pass
Login with valid credentials	JWT issued, redirect to dashboard	Pass
Login with wrong password	401 Unauthorized error	Pass
Fetch CBI quiz questions	13 questions with sections and options returned	Pass
Submit quiz with all 13 answers	Scores calculated, AI called, result saved	Pass
Submit quiz with missing answers	400 Validation error	Pass
Reverse scoring (question 10)	$\text{adjustedValue} = 100 - \text{rawValue}$	Pass
Mental fatigue score calculation	Correct value per Equation 4	Pass

Test Case	Expected Behaviour	Result
AI prediction returns valid burn rate	Value in [0, 1] range	Pass
Risk level mapping	Correct threshold applied	Pass
Alert generated on high risk	high_risk alert created	Pass
Alert generated on trend spike	trend_spike alert (delta $\geq$ 0.15)	Pass
Employee dashboard data	Profile, gauge, trend, section scores	Pass
Admin dashboard aggregation	Dept. stats, risk distribution, KPIs	Pass
Department drill-down with filters	Correct filtering and sorting	Pass
Department analytics charts	All chart data populated	Pass
Employee detail (admin view)	KPI and burn rate data displayed	Pass
Mark alert as read	Alert status updated, cache invalidated	Pass
Resource CRUD (create, update, delete)	Resource persisted, table refreshed	Pass
Route guard (employee accessing admin)	Redirect to employee dashboard	Pass
Route guard (unauthenticated access)	Redirect to login page	Pass
JWT expiry handling	401 response, redirect to login	Pass

All 23 test cases passed, confirming that the core workflows function as designed. Testing covered authentication, quiz scoring, AI integration, dashboard aggregation, alert generation, CRUD operations, and access control.

5.3 System Feature Evaluation

Table 8 evaluates each objective defined in Section 1.4 against the implemented system features.

Table 8: Evaluation of system objectives.

Obj.	Description	Status
1	Implement CBI-based assessment with mental fatigue scoring	Fully implemented
2	Train ML model for burn rate prediction	Achieved ($R^2 = 0.90$)
3	Build employee self-monitoring portal	Fully implemented (4 pages)
4	Build admin command centre with analytics	Fully implemented (6 pages)
5	Implement automated risk alerting	Fully implemented (3 alert types)
6	Demonstrate microservice architecture	Fully implemented (3 services)

All six objectives have been met. Comprehensive self-monitoring is provided by the employee portal, including gauge visualizations, trend charts, and support resources. Department-level heatmaps, risk distribution analytics, KPI correlation views, and a

fully functional alert inbox are offered by the admin portal. The ML concern is cleanly separated from the web backend by the microservice architecture.

5.4 Answering the Research Questions

RQ1: How effectively can a Random Forest regression model predict employee burn rate using only three input features?

An R^2 of 0.90 and an MAE of 0.048 are achieved by the Random Forest model. This demonstrates that three features: designation, resource allocation, and mental fatigue score are enough for effective burn rate prediction. Published results using the full feature set are comparable to the model's performance, which indicates that the dominant signal in the data is captured by the three selected features.

RQ2: Can a web-based system provide actionable burnout insights to both employees and HR administrators?

The employee portal provides personalised burn rate gauges, historical trend charts, section score breakdowns, and contextual help resources, enabling workers to monitor their own mental health proactively. The admin portal provides department heatmaps, risk distribution charts, KPI-vs-burn-rate correlations, and automated alerts, giving HR managers actionable visibility into organisational burnout patterns. The functional testing results confirm that all data flows correctly from quiz submission through prediction to dashboard visualisation.

RQ3: What are the practical benefits and trade-offs of using a microservice architecture to separate ML inference from the application backend?

The microservice architecture provides three concrete benefits: (1) technology heterogeneity, Python's scikit-learn ecosystem for ML and JavaScript's Express ecosystem for web logic are each used where they excel; (2) independent development, the AI

service can be retrained and redeployed without modifying or restarting the backend; (3) the mock/real switch (AI_USE MOCK) enables fully functional development without running the Python service. The primary trade-off is increased operational complexity: two servers must be running and communicating, and network failures between them require timeout handling and error propagation.

5.5 Comparison with Related Work

As shown in Table 1, this system is the only one among the reviewed works that brings together all four elements: a trained ML model, a deployed web application, organisational-level analytics, and a validated burnout instrument (CBI).

Pradhan and Jena’s framework [9] applies machine learning for burnout classification, but it does not include a web deployment or organisational dashboards. Commercial platforms like Culture Amp [29] provide web-based analytics, yet they depend on general-purpose engagement surveys rather than ML prediction. Kaggle community notebooks [15] show model training on the same dataset, but predictions are not integrated into a usable application.

This thesis addresses the integration gap by combining psychometric assessment, machine learning prediction, and multi-level analytics into a single platform that serves both individual workers and organisational decision makers.

CHAPTER VI

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This thesis presented the design, implementation, and evaluation of an intelligent web-based framework for the early detection of employee burnout in industrial workplaces. The system brings together a validated psychometric instrument (the Copenhagen Burnout Inventory), a machine learning prediction model (Random Forest regression), and a full-stack web application with role-based portals for both employees and administrators.

The Random Forest model, trained on approximately 22,000 records from a public dataset, achieves an R^2 of 0.90 using only three input features. This demonstrates that effective burn rate prediction is possible even with a minimal feature set. The employee portal equips workers with self-monitoring tools, including burn rate gauges, trend charts, and contextual support resources. Meanwhile, the admin portal offers organisational-level visibility through department heatmaps, risk distribution analytics, KPI correlations, and automated alerts.

The microservice architecture successfully separates the machine learning concern (Python/FastAPI) from the web application backend (Node.js/Express), which allows independent development and deployment. End-to-end functional testing confirmed that all core workflows operate correctly, from quiz submission and scoring through AI prediction to dashboard visualisation and alert generation.

6.2 Contributions

This thesis makes the following contributions:

1. **An integrated burnout detection platform** that combines psychometric assessment, machine learning prediction, and multi-level organisational analytics within a single web-based system. This combination is not found in any of the reviewed related works.
2. **A practical CBI-to-ML pipeline** that transforms 13-question Copenhagen Burnout Inventory responses into a continuous burn rate prediction through a clearly documented scoring and feature engineering process. This includes reverse scoring, section averaging, and normalisation.
3. **A demonstrable microservice pattern** for integrating machine learning into web applications, with a clean mock/real switch mechanism that enables independent development of the ML and web components.
4. **A comprehensive admin analytics suite** including department heatmaps, risk distribution charts, KPI-vs-burn-rate correlations, and automated three-type alert generation (high risk, critical risk, and trend spike).
5. **A complete design system and component library** with 15 reusable UI primitives and 12 data visualisation components, including a custom SVG burn rate gauge.

6.3 Achievement of Objectives

All six objectives defined in Section 1.4 have been fully achieved:

1. The CBI-based assessment is implemented as a two-step wizard with proper reverse scoring and mental fatigue score derivation.

2. The Random Forest model achieves $R^2 = 0.90$, $MAE = 0.048$, and $RMSE = 0.061$, significantly outperforming the Linear Regression baseline.
3. The employee portal includes four fully functional pages: dashboard, assessment, results, and history.
4. The admin portal includes six pages: command centre, department detail (with employees and analytics tabs), company-wide employee list, individual employee detail, alerts inbox, and resource management.
5. Three alert types (high risk, critical risk, and trend spike) are automatically generated and displayed in a filterable inbox with mark-read functionality.
6. The three-service architecture (React/Next.js, Node.js/Express, Python/FastAPI) is fully operational with mock/real mode switching.

6.4 Limitations

Several limitations should be acknowledged:

- **Simulated HR data.** Designation and resource allocation values are randomly generated at registration rather than sourced from a real factory HR system. While values are drawn from the training dataset's ranges, they do not reflect actual workplace conditions.
- **Limited feature set.** The model uses only three features. Predictive accuracy could be improved by including additional signals such as actual shift patterns, overtime records, or longitudinal CBI trends.
- **Single dataset.** Model training and evaluation rely on a single public Kaggle dataset. The model's performance may not generalise across different industries, cultures, or organisational contexts.

- **Simulated KPI data.** The employee KPI records (attendance, productivity, quality) are seeded rather than collected from real workplace systems, which limits the validity of the KPI-vs-burn-rate correlation analysis.
- **Desktop-only interface.** The application is designed for minimum 992 px screen widths. Factory workers using personal mobile devices cannot access the system.
- **No formal usability study.** The system was tested functionally but not evaluated through a structured usability study with real end users.
- **No real-time updates.** The admin dashboard uses polling-based data fetching via TanStack Query rather than real-time push notifications via WebSocket or Server-Sent Events.

6.5 Future Work

The following directions are proposed for future development:

- **Mobile responsive design.** Extend the frontend with responsive breakpoints to support mobile and tablet access, enabling factory workers to complete assessments on personal devices.
- **Real-time alerts.** Implement WebSocket or Server-Sent Events to push new alerts to the admin dashboard instantly, without requiring page refresh or polling intervals.
- **Integration with real HR systems.** Replace the simulated HR profile generator with connectors to actual HR information systems (e.g., SAP SuccessFactors, BambooHR) to provide authentic designation and workload data.
- **Advanced ML models.** Explore Gradient Boosting (XGBoost, LightGBM)

and time-series models that incorporate longitudinal CBI trends, enabling the system to predict burn rate trajectories rather than point estimates.

- **Expanded feature engineering.** Incorporate additional features such as actual overtime hours, shift rotation frequency, and seasonal patterns to improve model accuracy.
- **Export functionality.** Add CSV and PDF export for department analytics reports and assessment history, enabling offline review and reporting.
- **Accessibility improvements.** Add ARIA labels to custom components (gauge, heatmap) and conduct automated accessibility testing to ensure compliance with WCAG guidelines.
- **Multi-language support.** Internationalise the user interface to support Vietnamese and other languages relevant to the target industrial workforce.
- **Formal usability evaluation.** Conduct a structured usability study with factory workers and HR personnel to validate the system's effectiveness and identify user experience improvements.

LIST OF REFERENCES

- [1] World Health Organization, “Burn-out an ‘occupational phenomenon’: International Classification of Diseases,” WHO, May 2019. [Online]. Available: <https://www.who.int/news/item/28-05-2019-burn-out-an-occupational-phenomenon-international-classification>
- [2] C. Maslach, W. B. Schaufeli, and M. P. Leiter, “Job burnout,” *Annual Review of Psychology*, vol. 52, no. 1, pp. 397–422, 2001.
- [3] C. Maslach and S. E. Jackson, “The measurement of experienced burnout,” *Journal of Organizational Behavior*, vol. 2, no. 2, pp. 99–113, 1981.
- [4] H. J. Freudenberger, “Staff burn-out,” *Journal of Social Issues*, vol. 30, no. 1, pp. 159–165, 1974.
- [5] T. S. Kristensen, M. Borritz, E. Villadsen, and K. B. Christensen, “The Copenhagen Burnout Inventory: A new tool for the assessment of burnout,” *Work & Stress*, vol. 19, no. 3, pp. 192–207, 2005.
- [6] B. Wigert and S. Agrawal, “Employee burnout, Part 1: The 5 main causes,” Gallup Workplace, 2020. [Online]. Available: <https://www.gallup.com/workplace/237059/employee-burnout-part-main-causes.aspx>
- [7] K. Ahola, A. Toppinen-Tanner, and J. Seppänen, “Interventions to alleviate burnout symptoms and to support return to work among employees with burnout: System-

- atic review and meta-analysis,” *Burnout Research*, vol. 4, pp. 1–11, 2017.
- [8] R. Bianchi, I. S. Schonfeld, and E. Laurent, “Burnout: Moving beyond the status quo,” *International Journal of Stress Management*, vol. 26, no. 1, pp. 36–45, 2019.
- [9] R. K. Pradhan and L. K. Jena, “Employee performance at workplace: Conceptual model and empirical validation,” *Business Perspectives and Research*, vol. 5, no. 1, pp. 69–85, 2017.
- [10] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [11] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [12] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [13] A. Fernández-Delgado, E. Cernadas, S. Barro, and D. Amorim, “Do we need hundreds of classifiers to solve real world classification problems?” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 3133–3181, 2014.
- [14] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [15] V. Subhash, “Employee Burnout Prediction,” Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/vijaysubhashp/employee-burnout-prediction>

- [16] Meta Platforms, Inc., “React - A JavaScript library for building user interfaces,” 2024. [Online]. Available: <https://react.dev/>
- [17] Vercel, Inc., “Next.js - The React Framework,” 2024. [Online]. Available: <https://nextjs.org/>
- [18] OpenJS Foundation, “Node.js,” 2024. [Online]. Available: <https://nodejs.org/>
- [19] OpenJS Foundation, “Express - Fast, unopinionated, minimalist web framework for Node.js,” 2024. [Online]. Available: <https://expressjs.com/>
- [20] S. Ramírez, “FastAPI,” 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [21] T. Linsley, “TanStack Query (React Query),” 2024. [Online]. Available: <https://tanstack.com/query/latest>
- [22] Tailwind Labs, “Tailwind CSS,” 2024. [Online]. Available: <https://tailwindcss.com/>
- [23] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O’Reilly Media, 2015.
- [24] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. of California, Irvine, 2000.

- [25] Oracle Corporation, “MySQL 8.0 Reference Manual,” 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [26] MongoDB, Inc., “MongoDB Documentation,” 2024. [Online]. Available: <https://www.mongodb.com/docs/>
- [27] C. Burdo, “Zod - TypeScript-first schema validation,” 2024. [Online]. Available: <https://zod.dev/>
- [28] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015.
- [29] Culture Amp Pty Ltd, “Culture Amp - Employee Experience Platform,” 2024. [Online]. Available: <https://www.cultureamp.com/>
- [30] Recharts Group, “Recharts - A composable charting library built on React components,” 2024. [Online]. Available: <https://recharts.org/>

APPENDICES

APPENDIX A: COMPLETE API REFERENCE

Table 9 provides the complete list of RESTful endpoints exposed by the Express backend and the Python AI microservice.

Table 9: Complete API endpoint reference.

Method	Endpoint	Access	Purpose
<i>Authentication</i>			
POST	/api/auth/register	Public	Create account + HR profile
POST	/api/auth/login	Public	Authenticate, get JWT
GET	/api/auth/me	Any	Get current user profile
<i>Quiz</i>			
GET	/api/quiz	Employee	Fetch CBI quiz questions
POST	/api/quiz/submit	Employee	Submit answers, get prediction
GET	/api/quiz/history	Employee	Past submissions (paginated)
<i>Employee Dashboard</i>			
GET	/api/dashboard/employee	Employee	Full dashboard data
GET	/api/resources	Employee	Help resources for risk level
<i>Admin Dashboard</i>			
GET	/api/dashboard/admin	Admin	Command centre overview

Method	Endpoint	Access	Purpose
GET	/api/dashboard/admin/ department/:id	Admin	Department drill-down (sort/filter/paginate)
GET	/api/dashboard/admin/ department/:id/analytics	Admin	Department analytics
GET	/api/dashboard/admin/ employees	Admin	Company-wide employee list
GET	/api/dashboard/admin/ employees/:userId	Admin	Single employee detail
<i>Alerts</i>			
GET	/api/alerts	Admin	List alerts (filterable)
PATCH	/api/alerts/:id/read	Admin	Mark single alert as read
PATCH	/api/alerts/read-all	Admin	Mark all alerts as read
<i>Admin Resource Management</i>			
GET	/api/admin/resources	Admin	List all help resources
POST	/api/admin/resources	Admin	Create a help resource
PUT	/api/admin/resources/:id	Admin	Update a help resource
DELETE	/api/admin/resources/:id	Admin	Delete a help resource
<i>AI Microservice (internal)</i>			
POST	/api/predict	Internal	Predict burn rate
GET	/api/health	Internal	AI service health check

All responses use the JSON envelope format:

Success: { "success": true, "data": { ... } }

Error: { "success": false, "error": { "code": "...", "message": "..." } }

APPENDIX B: DATABASE SCHEMA

B.1 MySQL Tables

```
CREATE TABLE users (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    email         VARCHAR(255) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    first_name    VARCHAR(100) NOT NULL,  
    last_name     VARCHAR(100) NOT NULL,  
    gender        ENUM('Male','Female','Other') NOT NULL,  
    date_of_birth DATE,  
    role          ENUM('employee','admin') NOT NULL DEFAULT 'employee',  
    is_active     BOOLEAN NOT NULL DEFAULT TRUE,  
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
                ON UPDATE CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE departments (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    name          VARCHAR(100) UNIQUE NOT NULL,  
    location      VARCHAR(100),  
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```

CREATE TABLE hr_profiles (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    user_id           INT UNIQUE NOT NULL,
    department_id     INT NOT NULL,
    date_of_joining   DATE NOT NULL,
    company_type      ENUM('Service','Product') NOT NULL,
    wfh_available     BOOLEAN NOT NULL DEFAULT FALSE,
    designation       INT NOT NULL,
    resource_allocation INT,
    shift_type        ENUM('Day','Night','Rotating')
                    NOT NULL DEFAULT 'Day',
    created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                    ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (department_id)
        REFERENCES departments(id) ON DELETE CASCADE
);

```

```

CREATE TABLE assessment_results (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    user_id           INT NOT NULL,
    personal_burnout_score FLOAT,
    work_burnout_score    FLOAT,

```



```

    mental_fatigue_score    FLOAT NOT NULL,
    predicted_burn_rate     FLOAT NOT NULL,
    risk_level              ENUM('low','moderate','high','critical')
                           NOT NULL,
    taken_at                TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_user_history (user_id, taken_at)
);

```

```

CREATE TABLE alerts (
    id                      INT AUTO_INCREMENT PRIMARY KEY,
    assessment_result_id    INT NOT NULL,
    user_id                 INT NOT NULL,
    alert_type              ENUM('high_risk','critical_risk',
                               'trend_spike') NOT NULL,
    message                 TEXT NOT NULL,
    is_read                 BOOLEAN NOT NULL DEFAULT FALSE,
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (assessment_result_id)
        REFERENCES assessment_results(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_unread_alerts (is_read, created_at)
);

```

```

CREATE TABLE employee_kpis (

```

```

id            INT AUTO_INCREMENT PRIMARY KEY,
user_id       INT NOT NULL,
period_date   DATE NOT NULL,
attendance_rate  FLOAT NOT NULL,
productivity_score FLOAT NOT NULL,
overtime_hours  FLOAT NOT NULL,
tasks_completed INT NOT NULL,
quality_score  FLOAT NOT NULL,
days_absent   INT NOT NULL,
created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
UNIQUE INDEX uq_kpi_user_period (user_id, period_date),
INDEX idx_kpi_period (period_date)
);

```

```

CREATE TABLE help_resources (
id            INT AUTO_INCREMENT PRIMARY KEY,
title         VARCHAR(200) NOT NULL,
description    TEXT,
url           VARCHAR(500),
min_risk_level ENUM('low','moderate','high','critical') NOT NULL,
is_active     BOOLEAN NOT NULL DEFAULT TRUE,
created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
INDEX idx_active_risk (is_active, min_risk_level)
);

```

B.2 MongoDB Collections

Collection: cbi_questions - Singleton document containing the complete CBI quiz definition.

```
{
  "_id": ObjectId,
  "name": "Copenhagen Burnout Inventory (excerpt)",
  "version": 1,
  "isActive": true,
  "responseOptions": [
    {"label": "Always", "value": 100},
    {"label": "Often", "value": 75},
    {"label": "Sometimes", "value": 50},
    {"label": "Seldom", "value": 25},
    {"label": "Never/almost never", "value": 0}
  ],
  "sections": [
    {"id": "personal_burnout",
     "title": "Personal burnout",
     "questionIds": [1,2,3,4,5,6]},
    {"id": "work_related_burnout",
     "title": "Work-related burnout",
     "questionIds": [7,8,9,10,11,12,13]}
  ],
  "questions": [ /* 13 question objects */ ]
}
```

```
}
```

Collection: quiz_submissions - One document per assessment attempt.

```
{
  "_id": ObjectId,
  "mysqlAssessmentId": 42,
  "mysqlUserId": 1,
  "quizVersion": 1,
  "responses": [
    {"questionId": 1, "answerLabel": "Often",
     "answerValue": 75},
    {"questionId": 10, "answerLabel": "Seldom",
     "rawValue": 25, "adjustedValue": 75,
     "reverseScored": true},
    ...
  ],
  "personalBurnoutAvg": 58.33,
  "workBurnoutAvg": 64.29,
  "submittedAt": ISODate("2026-05-03T10:30:00Z")
}
```

APPENDIX C: COPENHAGEN BURNOUT INVENTORY QUESTIONNAIRE

The system uses 13 questions from the Copenhagen Burnout Inventory, covering two sections. Response options for all questions are: Always (100), Often (75), Sometimes (50), Seldom (25), Never/almost never (0).

C.1 Personal Burnout (Questions 1–6)

1. How often do you feel tired?
2. How often are you physically exhausted?
3. How often are you emotionally exhausted?
4. How often do you think: “I can’t take it anymore”?
5. How often do you feel worn out?
6. How often do you feel weak and susceptible to illness?

C.2 Work-Related Burnout (Questions 7–13)

7. Do you feel worn out at the end of the working day?
8. Are you exhausted in the morning at the thought of another day at work?
9. Do you feel that every working hour is tiring for you?
10. Do you have enough energy for family and friends during leisure time?
(Reverse-scored: $adjustedValue = 100 - rawValue$)
11. Is your work emotionally exhausting?
12. Does your work frustrate you?
13. Do you feel burnt out because of your work?

C.3 Scoring

1. Calculate the personal burnout average: mean of questions 1–6 (range 0–100).
2. Calculate the work-related burnout average: mean of questions 7–13 (range 0–100), with question 10 reverse-scored.
3. Calculate the mental fatigue score: $\text{mentalFatigueScore} = \frac{\text{personalAvg} + \text{workAvg}}{20}$ (range 0.0–10.0).